This is a project on classifying the message recieved through a chatbot into severity categories for proper judgement of the accident scenarios.

Importing the required Libraries

```
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.10/dist-packages (1.5.16)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.10/dist-packages (from kaggle) (1.16.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from kaggle) (2023.11.17)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.8.2)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.31.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from kaggle) (4.66.1)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.10/dist-packages (from kaggle) (8.0.1)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.0.7)
Requirement already satisfied: bleach in /usr/local/lib/python3.10/dist-packages (from kaggle) (6.1.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.10/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.10/dist-packages (from python-slugify->kaggle) (1.3
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.3.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.6)
```

```
!pip install nlpaug
```

```
Requirement already satisfied: nlpaug in /usr/local/lib/python3.10/dist-packages (1.1.11)
Requirement already satisfied: numpy>=1.16.2 in /usr/local/lib/python3.10/dist-packages (from nlpaug) (1.23.5)
Requirement already satisfied: pandas>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from nlpaug) (1.5.3)
Requirement already satisfied: requests>=2.22.0 in /usr/local/lib/python3.10/dist-packages (from nlpaug) (2.31.0)
Requirement already satisfied: gdown>=4.0.0 in /usr/local/lib/python3.10/dist-packages (from nlpaug) (4.6.6)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from gdown>=4.0.0->nlpaug) (3.13.1)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from gdown>=4.0.0->nlpaug) (1.16.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from gdown>=4.0.0->nlpaug) (4.66.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.10/dist-packages (from gdown>=4.0.0->nlpaug) (4.11.2)
Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.2.0->nlpaug) (2
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.2.0->nlpaug) (2023.3.post
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>=2.22.0->nlpau
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.22.0->nlpaug) (3.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>=2.22.0->nlpaug) (2.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>=2.22.0->nlpaug) (20
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.10/dist-packages (from beautifulsoup4->gdown>=4.0.0->nlpa
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.10/dist-packages (from requests>=2.22.0->nlpaug)
```

```
!pip install torch
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.10/dist-packages (2.1.0+cu121)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch) (3.13.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages (from torch) (4.5.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch) (1.12)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch) (3.2.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch) (3.1.2)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch) (2023.6.0)
Requirement already satisfied: triton==2.1.0 in /usr/local/lib/python3.10/dist-packages (from torch) (2.1.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch) (2.1.3)
Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.10/dist-packages (from sympy->torch) (1.3.0)
```

```
!pip install torchtext
```

```
Requirement already satisfied: torchtext in /usr/local/lib/python3.10/dist-packages (0.16.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from torchtext) (4.66.1)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from torchtext) (2.31.0)
Requirement already satisfied: torch==2.1.0 in /usr/local/lib/python3.10/dist-packages (from torchtext) (2.1.0+cu121)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from torchtext) (1.23.5)
Requirement already satisfied: torchdata==0.7.0 in /usr/local/lib/python3.10/dist-packages (from torchtext) (0.7.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch==2.1.0->torchtext) (3.13.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages (from torch==2.1.0->torchtext) (4.5.
Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch==2.1.0->torchtext) (1.12)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch==2.1.0->torchtext) (3.2.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch==2.1.0->torchtext) (3.1.2)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch==2.1.0->torchtext) (2023.6.0)
Requirement already satisfied: triton==2.1.0 in /usr/local/lib/python3.10/dist-packages (from torch==2.1.0->torchtext) (2.1.0)
Requirement already satisfied: urllib3>=1.25 in /usr/local/lib/python3.10/dist-packages (from torchdata==0.7.0->torchtext) (2.0.
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->torchtext) (3
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->torchtext) (3.6)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->torchtext) (2023.11
```

```
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch==2.1.0->torchtext)
Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.10/dist-packages (from sympy->torch==2.1.0->torchtext) (1.
```

```
!pip install unidecode
```

```
Requirement already satisfied: unidecode in /usr/local/lib/python3.10/dist-packages (1.3.7)
```

```
!pip install autocorrect
```

```
Requirement already satisfied: autocorrect in /usr/local/lib/python3.10/dist-packages (2.6.1)
```

```
!pip install --upgrade tensorflow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.10/dist-packages (2.15.0.post1)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=23.5.26 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (23.5.26)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: h5py>=2.9.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (3.9.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (16.0.6)
Requirement already satisfied: ml-dtypes~=0.2.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: numpy<2.0.0,>=1.23.5 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (1.23.5)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from tensorflow) (23.2)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<5.0.0dev,>=3.20.3 in /usr/local/li
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from tensorflow) (67.7.2)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (2.4.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (4.5.0)
Requirement already satisfied: wrapt<1.15,>=1.11.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (1.14.1)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (1.60.0)
Requirement already satisfied: tensorboard<2.16,>=2.15 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (2.15.1)
Requirement already satisfied: tensorflow-estimator<2.16,>=2.15.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (
Requirement already satisfied: keras<2.16,>=2.15.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow) (2.15.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from astunparse>=1.6.0->tensorflow
Requirement already satisfied: google-auth<3,>=1.6.3 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->t
Requirement already satisfied: google-auth-oauthlib<2,>=0.5 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->tensorf
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->ten
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from tensorboar
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorboard<2.16,>=2.15->tensorf
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->te
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->ten
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.10/dist-packages (from google-auth<3,>=1.6.3->tensorboard
Requirement already satisfied: requests-oauthlib>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from google-auth-oauthlib<2,
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorboard<2.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorbo
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0->tensorbo
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1->tensorboard<2
Requirement already satisfied: pyasn1<0.6.0,>=0.4.6 in /usr/local/lib/python3.10/dist-packages (from pyasn1-modules>=0.2.1->goog
Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.10/dist-packages (from requests-oauthlib>=0.7.0->google
```

```
import zipfile as zf
import pandas as pd
import numpy as np
import datetime as dt
import re
import string
import spacy
import tensorflow as tf
import torch

import matplotlib.pyplot as plt
%matplotlib inline
from matplotlib.ticker import MaxNLocator
import seaborn as sns
from statsmodels.graphics.mosaicplot import mosaic
from scipy.sparse import vstack

from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score
from imblearn.over_sampling import SMOTE

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.util import ngrams
from nltk import pos_tag
from collections import Counter
from gensim.models import Word2Vec

from keras.models import Model, Sequential
from keras.optimizers import Adam
from keras.layers import Input, Embedding, LSTM, Dense, concatenate, SimpleRNN, Bidirectional
from keras.regularizers import l2
from keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

import unicodedata
from unidecode import unidecode
from autocorrect import Speller
from string import punctuation
from wordcloud import WordCloud, STOPWORDS

import nlpaug.augmenter.word as naw
from nlpaug.util import Action
from transformers import pipeline
import torchtext.utils as utils
import torchtext.vocab as vocab

import warnings
warnings.filterwarnings("ignore")
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
True
```

PROBLEM CONTEXT:

The database comes from one of the biggest industry in Brazil and in the world. It is an urgent need for industries/companies around the globe to understand why employees still suffer some injuries/accidents in plants. Sometimes they also die in such environment.

DATA DESCRIPTION:

This The database is basically records of accidents from 12 different plants in 03 different countries which every line in the data is an occurrence of an accident.

Columns description:

- Data: timestamp or time/date information
- Countries: which country the accident occurred (anonymised)
- Local: the city where the manufacturing plant is located (anonymised)
- Industry sector: which sector the plant belongs to
- Accident level: from I to VI, it registers how severe was the accident (I means not severe but VI means very severe)
- Potential Accident Level: Depending on the Accident Level, the database also registers how severe the accident could have been (due to other factors involved in the accident)
- Genre: if the person is male of female

- Employee or Third Party: if the injured person is an employee or a third party
- Critical Risk: some description of the risk involved in the accident
- Description: Detailed description of how the accident happened.

PROJECT OBJECTIVE

Design a ML/DL based chatbot utility which can help the professionals to highlight the safety risk as per the incident description.

MILESTONE 1

Step 1: Import the data

```
!kaggle datasets download -d ihmstefanini/industrial-safety-and-health-analytics-database
```

```
Traceback (most recent call last):
  File "/usr/local/bin/kaggle", line 5, in <module>
    from kaggle.cli import main
  File "/usr/local/lib/python3.10/dist-packages/kaggle/__init__.py", line 23, in <module>
    api.authenticate()
  File "/usr/local/lib/python3.10/dist-packages/kaggle/api/kaggle_api_extended.py", line 403, in authenticate
    raise IOError('Could not find {}. Make sure it\'s located in'
OSError: Could not find kaggle.json. Make sure it's located in /root/.kaggle. Or use the environment method.
```

```
folder = '/content/'
```

```
data = pd.read_csv(folder + 'Industrial_Safety_Data.csv')
pd.set_option('display.max_colwidth', None)
```

```
data.head()
```

	Unnamed: 0	Data	Countries	Local	Industry Sector	Accident Level	Potential Accident Level	Genre	Employee or Third Party	Critical Risk	Description
0	0	01-01-2016 00:00	Country_01	Local_01	Mining	I	IV	Male	Third Party	Pressed	While removing the drill rod of the Jumbo 08 for maintenance, the supervisor proceeds to loosen the support of the intermediate centralizer to facilitate the removal, seeing this the mechanic supports one end on the drill of the equipment to pull with both hands the bar and accelerate the removal from this, at this moment the bar slides from its point of support and tightens the fingers of the mechanic between the drilling bar and the beam of the jumbo. During the activation of a

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 425 entries, 0 to 424
Data columns (total 11 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
```

```
 0   Unnamed: 0                425 non-null    int64
 1   Data                      425 non-null    object
 2   Countries                 425 non-null    object
 3   Local                     425 non-null    object
 4   Industry Sector           425 non-null    object
 5   Accident Level            425 non-null    object
 6   Potential Accident Level  425 non-null    object
 7   Genre                     425 non-null    object
 8   Employee or Third Party   425 non-null    object
 9   Critical Risk             425 non-null    object
 10  Description               425 non-null    object
dtypes: int64(1), object(10)
memory usage: 36.6+ KB
```

```
data.shape
```

```
(425, 11)
```

Step 2: Data cleansing

```
data.isna().sum()
```

```
Unnamed: 0                  0
Data                        0
Countries                   0
Local                       0
Industry Sector             0
Accident Level              0
Potential Accident Level    0
Genre                       0
Employee or Third Party     0
Critical Risk               0
Description                 0
dtype: int64
```

```
# Remove the duplicate records in the data

print('There are ', data.shape[0], 'records and ', data.duplicated().sum(), ' duplicates in this dataset.')
data = data.drop_duplicates()
print('There are ', data.shape[0], 'records and ', data.duplicated().sum(), ' duplicates in this dataset.')
```

```
There are  425 records and  0  duplicates in this dataset.
There are  425 records and  0  duplicates in this dataset.
```

```
# Remove the first 'Unnamed' column
data.drop(data.columns[0], axis=1, inplace=True)
```

```
# Renaming the columns appropriately
data.rename(columns={'Data':'Date', 'Countries':'Country', 'Local':'City', 'Genre':'Gender', 'Employee or Third Party':'Emp
```

```
# Extracting year & month from 'Data' column
data['Date'] = pd.to_datetime(data['Date'], format='%d-%m-%Y %H:%M')
data['Year'] = data['Date'].dt.year
data['Month'] = data['Date'].dt.month
data['Day'] = data['Date'].dt.day_name()
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 425 entries, 0 to 424
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Date                      425 non-null    datetime64[ns]
 1   Country                   425 non-null    object
 2   City                      425 non-null    object
 3   Industry Sector           425 non-null    object
 4   Accident Level            425 non-null    object
 5   Potential Accident Level  425 non-null    object
 6   Gender                    425 non-null    object
 7   Employee Type             425 non-null    object
 8   Critical Risk             425 non-null    object
 9   Description               425 non-null    object
 10  Year                      425 non-null    int64
 11  Month                     425 non-null    int64
 12  Day                       425 non-null    object
```

```
dtypes: datetime64[ns](1), int64(2), object(10)
memory usage: 46.5+ KB
```

```
data.head()
```

Date	Country	City	Industry Sector	Accident Level	Potential Accident Level	Gender	Employee Type	Critical Risk	Description	Year	Month	Day
									While removing the drill rod of the Jumbo 08 for maintenance, the supervisor proceeds to loosen the support of the intermediate centralizer to facilitate the removal, seeing this the mechanic			

```
# Checking the unique values of each column
col_ls = data.columns[:data.shape[1]-1]

for col in col_ls:
    print("\n", col, ":")
    print(data[col].unique())
```

```
 Date :
['2016-01-01T00:00:00.000000000' '2016-01-02T00:00:00.000000000'
 '2016-01-06T00:00:00.000000000' '2016-01-08T00:00:00.000000000'
 '2016-01-10T00:00:00.000000000' '2016-01-12T00:00:00.000000000'
 '2016-01-16T00:00:00.000000000' '2016-01-17T00:00:00.000000000'
 '2016-01-19T00:00:00.000000000' '2016-01-26T00:00:00.000000000'
 '2016-01-28T00:00:00.000000000' '2016-01-30T00:00:00.000000000'
 '2016-02-01T00:00:00.000000000' '2016-02-02T00:00:00.000000000'
 '2016-02-04T00:00:00.000000000' '2016-02-06T00:00:00.000000000'
 '2016-02-07T00:00:00.000000000' '2016-02-08T00:00:00.000000000'
 '2016-02-21T00:00:00.000000000' '2016-02-25T00:00:00.000000000'
 '2016-02-09T00:00:00.000000000' '2016-02-10T00:00:00.000000000'
 '2016-02-15T00:00:00.000000000' '2016-02-14T00:00:00.000000000'
 '2016-02-13T00:00:00.000000000' '2016-02-16T00:00:00.000000000'
 '2016-02-17T00:00:00.000000000' '2016-02-19T00:00:00.000000000'
 '2016-02-20T00:00:00.000000000' '2016-02-18T00:00:00.000000000'
 '2016-02-22T00:00:00.000000000' '2016-02-24T00:00:00.000000000'
 '2016-02-29T00:00:00.000000000' '2016-02-26T00:00:00.000000000'
 '2016-02-27T00:00:00.000000000' '2016-03-02T00:00:00.000000000'
 '2016-03-03T00:00:00.000000000' '2016-03-04T00:00:00.000000000'
 '2016-03-05T00:00:00.000000000' '2016-03-06T00:00:00.000000000'
 '2016-03-09T00:00:00.000000000' '2016-03-11T00:00:00.000000000'
 '2016-03-13T00:00:00.000000000' '2016-03-12T00:00:00.000000000'
 '2016-03-14T00:00:00.000000000' '2016-03-16T00:00:00.000000000'
 '2016-03-10T00:00:00.000000000' '2016-03-17T00:00:00.000000000'
 '2016-03-18T00:00:00.000000000' '2016-03-19T00:00:00.000000000'
```

```
 '2016-03-22T00:00:00.000000000' '2016-03-25T00:00:00.000000000'
 '2016-03-30T00:00:00.000000000' '2016-03-31T00:00:00.000000000'
 '2016-04-01T00:00:00.000000000' '2016-04-03T00:00:00.000000000'
 '2016-04-02T00:00:00.000000000' '2016-03-24T00:00:00.000000000'
 '2016-04-04T00:00:00.000000000' '2016-04-05T00:00:00.000000000'
 '2016-04-07T00:00:00.000000000' '2016-04-08T00:00:00.000000000'
 '2016-04-11T00:00:00.000000000' '2016-04-14T00:00:00.000000000'
 '2016-04-16T00:00:00.000000000' '2016-04-15T00:00:00.000000000'
 '2016-04-17T00:00:00.000000000' '2016-04-18T00:00:00.000000000'
 '2016-04-21T00:00:00.000000000' '2016-04-22T00:00:00.000000000'
 '2016-04-23T00:00:00.000000000' '2016-04-26T00:00:00.000000000'
 '2016-04-28T00:00:00.000000000' '2016-04-29T00:00:00.000000000'
 '2016-04-30T00:00:00.000000000' '2016-05-01T00:00:00.000000000'
 '2016-05-02T00:00:00.000000000' '2016-05-04T00:00:00.000000000'
 '2016-05-03T00:00:00.000000000' '2016-05-05T00:00:00.000000000'
 '2016-05-11T00:00:00.000000000' '2016-05-12T00:00:00.000000000'
 '2016-05-14T00:00:00.000000000' '2016-05-17T00:00:00.000000000'
 '2016-05-19T00:00:00.000000000' '2016-05-18T00:00:00.000000000'
 '2016-05-22T00:00:00.000000000' '2016-05-20T00:00:00.000000000'
 '2016-05-24T00:00:00.000000000' '2016-05-25T00:00:00.000000000'
 '2016-05-27T00:00:00.000000000' '2016-05-26T00:00:00.000000000'
 '2016-06-01T00:00:00.000000000' '2016-06-02T00:00:00.000000000'
 '2016-06-03T00:00:00.000000000' '2016-06-04T00:00:00.000000000'
 '2016-06-05T00:00:00.000000000' '2016-06-08T00:00:00.000000000'
 '2016-06-07T00:00:00.000000000' '2016-06-10T00:00:00.000000000'
 '2016-06-13T00:00:00.000000000' '2016-06-16T00:00:00.000000000'
 '2016-06-18T00:00:00.000000000' '2016-06-17T00:00:00.000000000'
 '2016-06-19T00:00:00.000000000' '2016-06-21T00:00:00.000000000'
 '2016-06-22T00:00:00.000000000' '2016-06-23T00:00:00.000000000'
 '2016-06-24T00:00:00.000000000' '2016-06-29T00:00:00.000000000'
```

```
data[data['Critical Risk']=='\nNot applicable'].head()
```

Date	Country	City	Industry Sector	Accident Level	Potential Accident Level	Gender	Employee Type	Critical Risk	Description	Year	Month	Day
									At approximately 5:45 pm, the			

```
temp = data
temp['Critical Risk'] = temp['Critical Risk'].str.replace(r'\nNot applicable', 'Not applicable', regex=True)
```

```
temp[temp['Critical Risk']=='Not applicable'].head()
```

Date	Country	City	Industry Sector	Accident Level	Potential Accident Level	Gender	Employee Type	Critical Risk	Description	Year	Month	Day
									At approximately 5:45 pm, the			

```
col_ls = data.columns[:data.shape[1]-1]

for col in col_ls:
    print("\n\n", col, ":")
    print("--------------------------------------------------------------------------------")
    print(data[col].unique())
```

```
 Date :
--------------------------------------------------------------------------------
['2016-01-01T00:00:00.000000000' '2016-01-02T00:00:00.000000000'
 '2016-01-06T00:00:00.000000000' '2016-01-08T00:00:00.000000000'
 '2016-01-10T00:00:00.000000000' '2016-01-12T00:00:00.000000000'
 '2016-01-16T00:00:00.000000000' '2016-01-17T00:00:00.000000000'
 '2016-01-19T00:00:00.000000000' '2016-01-26T00:00:00.000000000'
 '2016-01-28T00:00:00.000000000' '2016-01-30T00:00:00.000000000'
```

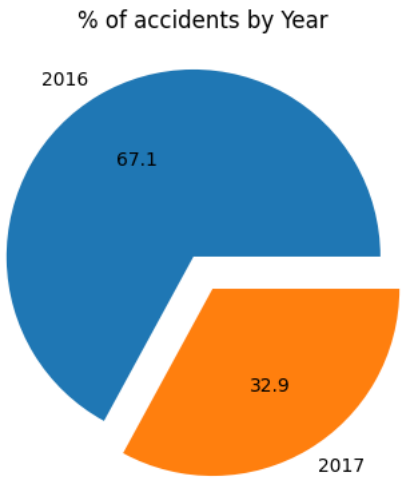
```
'2016-02-01T00:00:00.000000000' '2016-02-02T00:00:00.000000000'
'2016-02-04T00:00:00.000000000' '2016-02-06T00:00:00.000000000'
'2016-02-07T00:00:00.000000000' '2016-02-08T00:00:00.000000000'
'2016-02-21T00:00:00.000000000' '2016-02-25T00:00:00.000000000'
'2016-02-09T00:00:00.000000000' '2016-02-10T00:00:00.000000000'
'2016-02-15T00:00:00.000000000' '2016-02-14T00:00:00.000000000'
'2016-02-13T00:00:00.000000000' '2016-02-16T00:00:00.000000000'
'2016-02-17T00:00:00.000000000' '2016-02-19T00:00:00.000000000'
'2016-02-20T00:00:00.000000000' '2016-02-18T00:00:00.000000000'
'2016-02-22T00:00:00.000000000' '2016-02-24T00:00:00.000000000'
'2016-02-29T00:00:00.000000000' '2016-02-26T00:00:00.000000000'
'2016-02-27T00:00:00.000000000' '2016-03-02T00:00:00.000000000'
'2016-03-03T00:00:00.000000000' '2016-03-04T00:00:00.000000000'
'2016-03-05T00:00:00.000000000' '2016-03-06T00:00:00.000000000'
'2016-03-09T00:00:00.000000000' '2016-03-11T00:00:00.000000000'
'2016-03-13T00:00:00.000000000' '2016-03-12T00:00:00.000000000'
'2016-03-14T00:00:00.000000000' '2016-03-16T00:00:00.000000000'
'2016-03-10T00:00:00.000000000' '2016-03-17T00:00:00.000000000'
'2016-03-18T00:00:00.000000000' '2016-03-19T00:00:00.000000000'
'2016-03-22T00:00:00.000000000' '2016-03-25T00:00:00.000000000'
'2016-03-30T00:00:00.000000000' '2016-03-31T00:00:00.000000000'
'2016-04-01T00:00:00.000000000' '2016-04-03T00:00:00.000000000'
'2016-04-02T00:00:00.000000000' '2016-03-24T00:00:00.000000000'
'2016-04-04T00:00:00.000000000' '2016-04-05T00:00:00.000000000'
'2016-04-07T00:00:00.000000000' '2016-04-08T00:00:00.000000000'
'2016-04-11T00:00:00.000000000' '2016-04-14T00:00:00.000000000'
'2016-04-16T00:00:00.000000000' '2016-04-15T00:00:00.000000000'
'2016-04-17T00:00:00.000000000' '2016-04-18T00:00:00.000000000'
'2016-04-21T00:00:00.000000000' '2016-04-22T00:00:00.000000000'
'2016-04-23T00:00:00.000000000' '2016-04-26T00:00:00.000000000'
'2016-04-28T00:00:00.000000000' '2016-04-29T00:00:00.000000000'
'2016-04-30T00:00:00.000000000' '2016-05-01T00:00:00.000000000'
'2016-05-02T00:00:00.000000000' '2016-05-04T00:00:00.000000000'
'2016-05-03T00:00:00.000000000' '2016-05-05T00:00:00.000000000'
'2016-05-11T00:00:00.000000000' '2016-05-12T00:00:00.000000000'
'2016-05-14T00:00:00.000000000' '2016-05-17T00:00:00.000000000'
'2016-05-19T00:00:00.000000000' '2016-05-18T00:00:00.000000000'
'2016-05-22T00:00:00.000000000' '2016-05-20T00:00:00.000000000'
'2016-05-24T00:00:00.000000000' '2016-05-25T00:00:00.000000000'
'2016-05-27T00:00:00.000000000' '2016-05-26T00:00:00.000000000'
'2016-06-01T00:00:00.000000000' '2016-06-02T00:00:00.000000000'
'2016-06-03T00:00:00.000000000' '2016-06-04T00:00:00.000000000'
'2016-06-05T00:00:00.000000000' '2016-06-08T00:00:00.000000000'
'2016-06-07T00:00:00.000000000' '2016-06-10T00:00:00.000000000'
'2016-06-13T00:00:00.000000000' '2016-06-16T00:00:00.000000000'
'2016-06-18T00:00:00.000000000' '2016-06-17T00:00:00.000000000'
'2016-06-19T00:00:00.000000000' '2016-06-21T00:00:00.000000000'
```

Exploratory Data Analysis

UNIVARIATE ANALYSIS

```
def plot_uni(df, col, title):
    idx = df[col].value_counts().index
    plt.figure(figsize=(12,10))
    plt.subplot(2, 2, 1)
    plt.pie(df[col].value_counts(), autopct='%.1f', labels=idx, explode=[0.1]*len(idx))
    plt.title(title)
    plt.show()
```

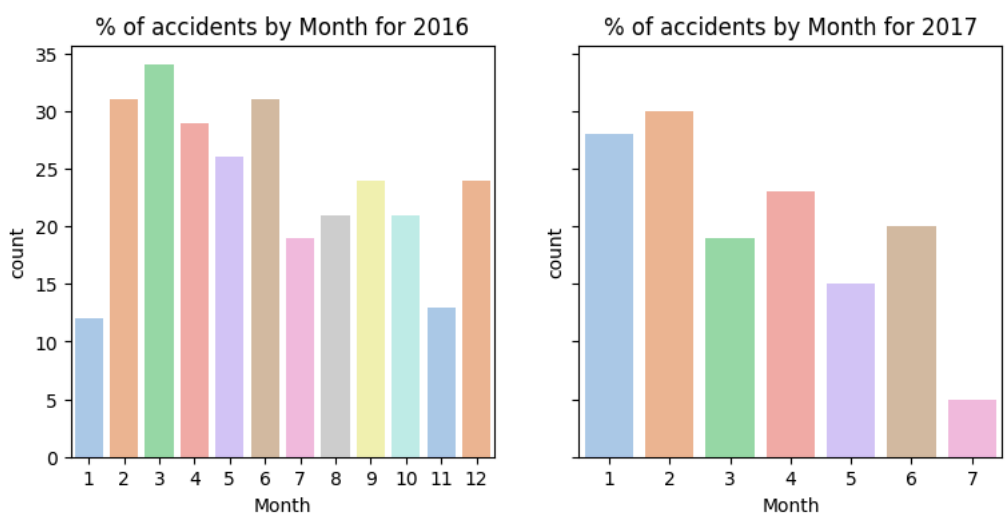
```
plot_uni(data, 'Year', '% of accidents by Year')
```



Since the data is available only until 9th July 2017, the numbers above do not provide enough evidence to confirm if accidents increased/ decreased in 2017.

Now, comparing the first half of both years, we can see that the accidents reduced by about 20% in 2017.

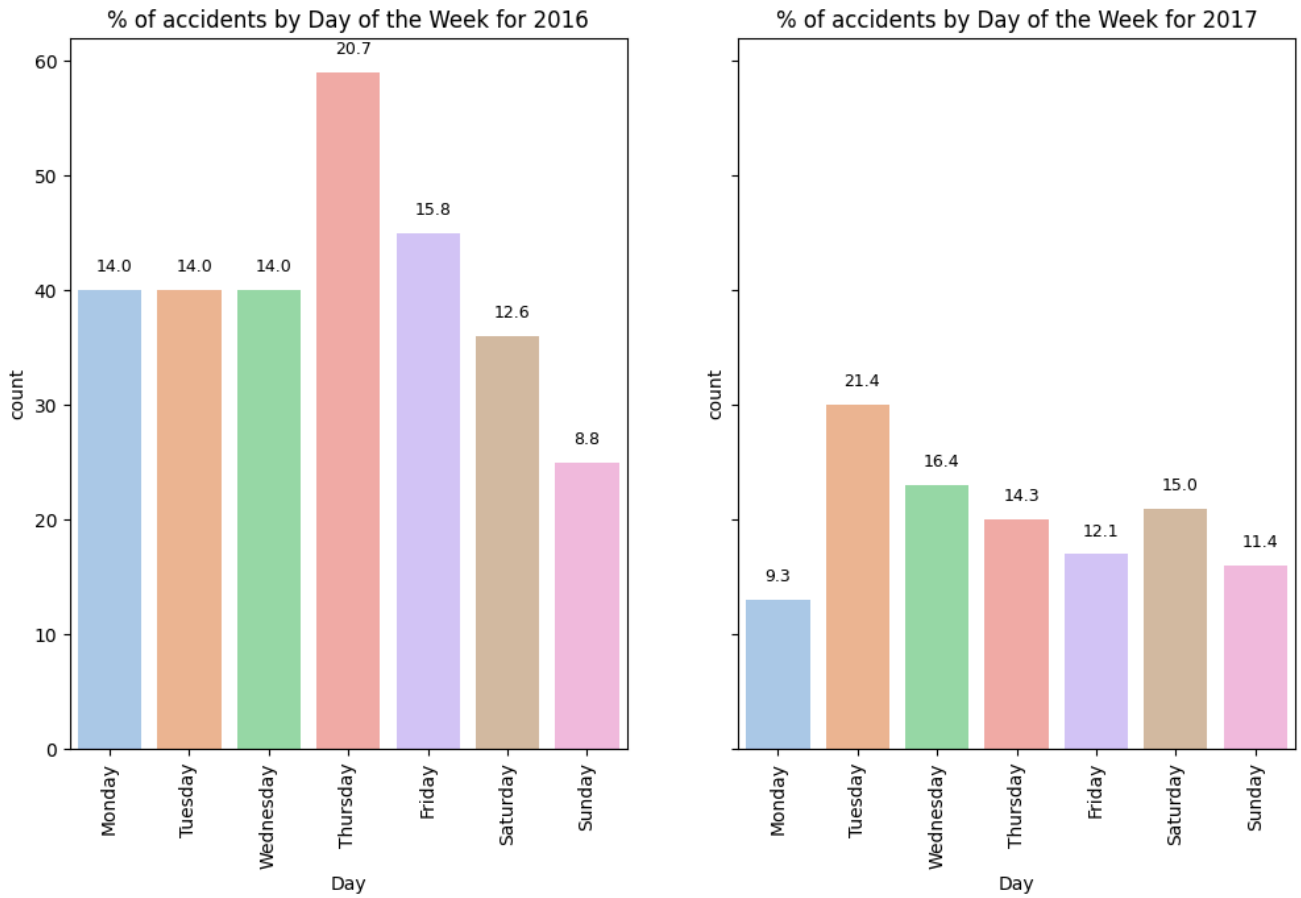
```
fig, axes = plt.subplots(1, 2, figsize=(9, 4), sharey=True)
# plt.subplot(1, 2, 2)
sns.countplot(data=data[data['Year']==2016], x='Month', palette='pastel', ax=axes[0])
axes[0].set_title('% of accidents by Month for 2016')
# plt.subplot(2, 2, 2)
sns.countplot(data=data[data['Year']==2017], x='Month', palette='pastel', ax=axes[1])
axes[1].set_title('% of accidents by Month for 2017')
plt.show()
```



You can see for each month from Feb - Jun, the number of accidents have reduced in 2017.

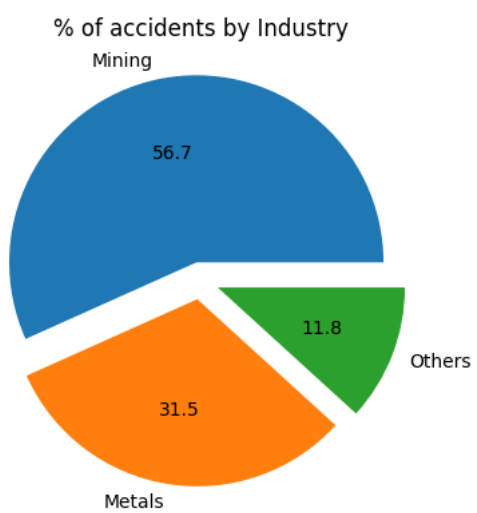
```
fig, axes = plt.subplots(1, 2, figsize=(12, 7), sharey=True)
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
data_16 = data[data['Year']==2016]
data_17 = data[data['Year']==2017]

sns.countplot(data=data_16, x='Day', order=day_order, palette='pastel', ax=axes[0])
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=90)
total = sum(data_16['Day'].value_counts())
for p in axes[0].patches:
    axes[0].annotate('{:.1f}'.format(100 * p.get_height() / total),
                (p.get_x(), p.get_height()), size=9, xytext=(10, 10), textcoords='offset points')
axes[0].set_title('% of accidents by Day of the Week for 2016')

```

```
sns.countplot(data=data_17, x='Day', order=day_order, palette='pastel', ax=axes[1])
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=90)
total = sum(data_17['Day'].value_counts())
for p in axes[1].patches:
    axes[1].annotate('{:.1f}'.format(100 * p.get_height() / total),
                (p.get_x(), p.get_height()), size=9, xytext=(10, 10), textcoords='offset points')
axes[1].set_title('% of accidents by Day of the Week for 2017')
plt.show()
```



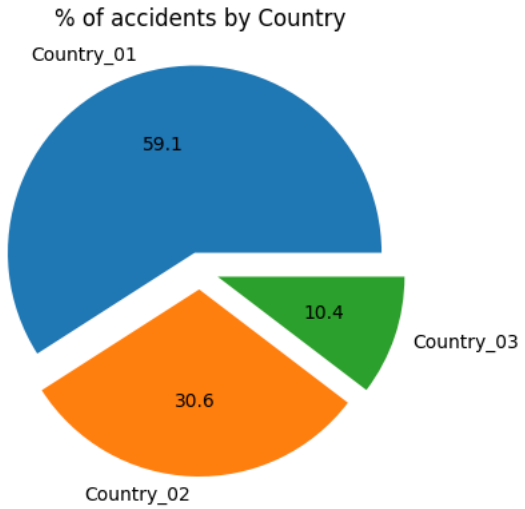
For Both 2016 & 2017, about 70-80% accidents have happened during the weekdays. People travelling for work on weekdays could be the possible reason for this.

```
plot_uni(data, 'Industry Sector', '% of accidents by Industry')
```



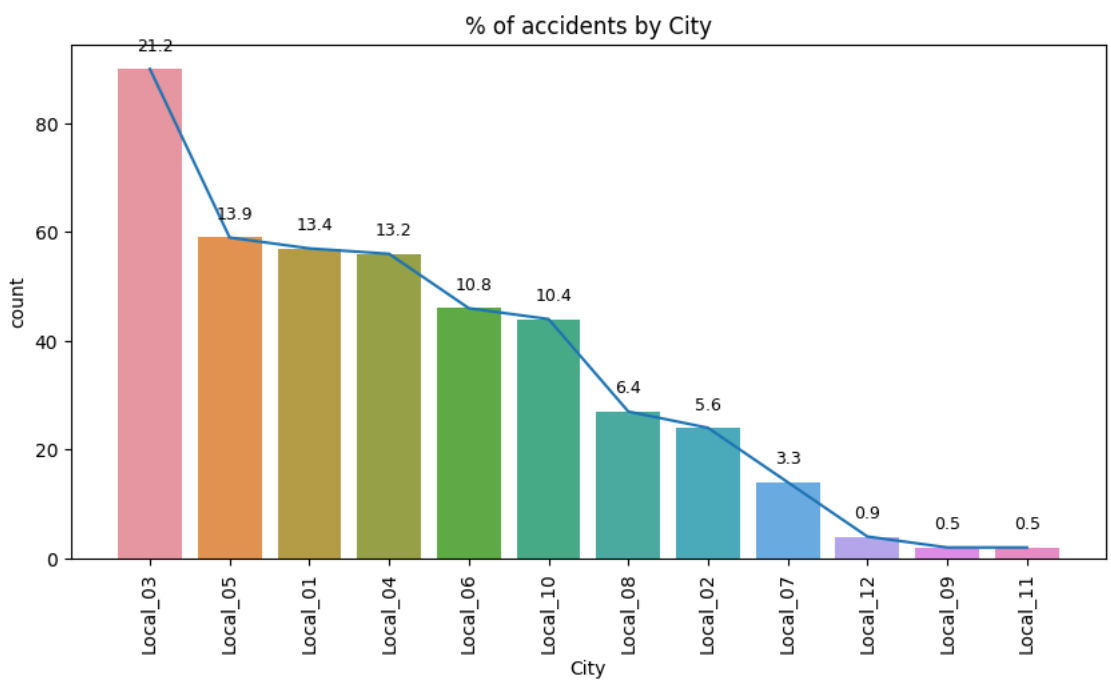
Mining industry witnesses the highest number of accidents, followed by Metals industry

```
plot_uni(data, 'Country', '% of accidents by Country')
```



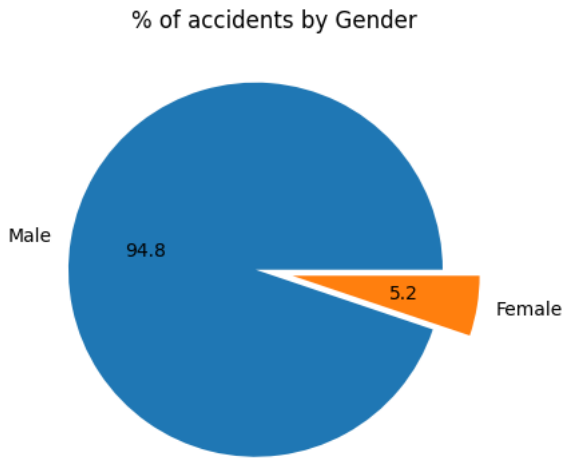
Country_01 witnesses the highest number of accidents, followed by Country02, which together contribute to ~90% of the total accidents.

```
plt.figure(figsize=(10, 5))
ax = sns.countplot(data, x='City', order=data['City'].value_counts().index)
sns.lineplot(data['City'].value_counts().values)
plt.xticks(rotation=90)
total = sum(data['City'].value_counts())
for p in ax.patches:
    ax.annotate('{:.1f}'.format(100 * p.get_height() / total),
                (p.get_x(), p.get_height()), size=9, xytext=(10, 10), textcoords='offset points')
plt.title('% of accidents by City')
plt.show()
```



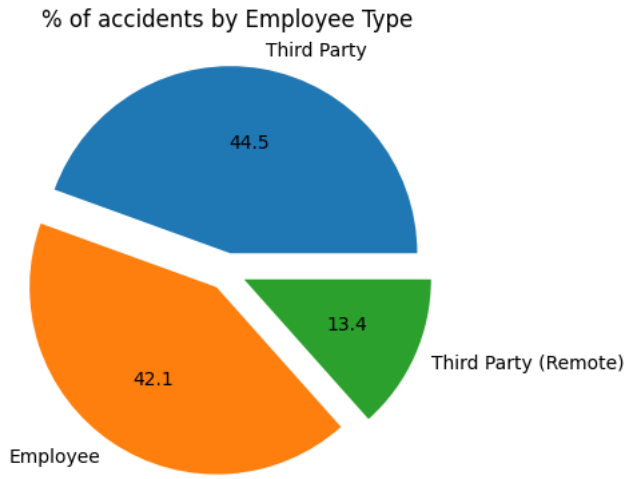
About 61% of the accidents together occur in plants located in Local_01, Local_03, Local_04 & Local_05 cities, of which Local_03 alone has recorded 21% accidents. Another 21% accidents are recorded by Local_06 & Local_10 cities. The bottom most 6 cities - Local_02, Local_07, Local_08, Local_09, Local_11, Local_12 cities witnessed only 17% of the accidents.

```
plot_uni(data, 'Gender', '% of accidents by Gender')
```



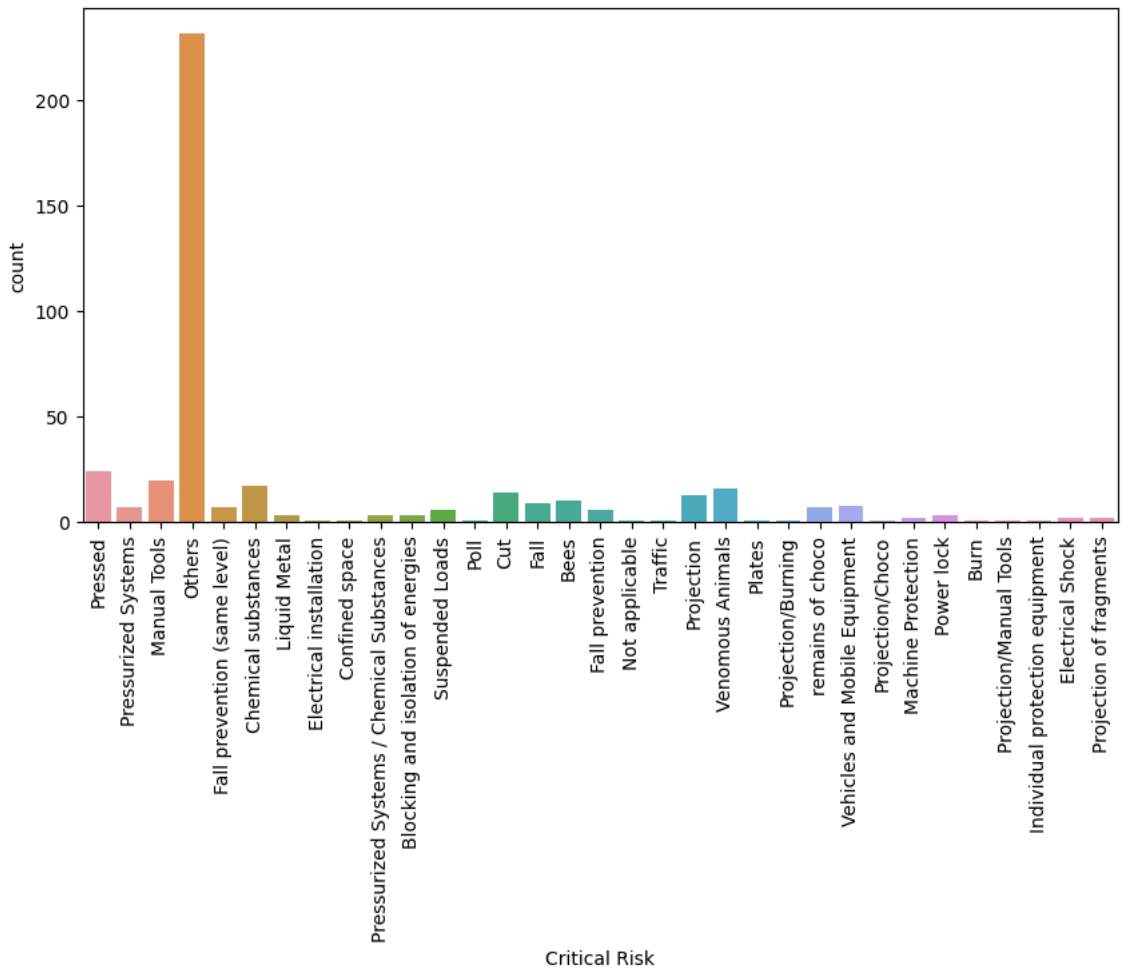
We can see the only ~5% Females have encountered accidents. This could be due to a general assumption of majority of industrial jobs being taken up by Males.

```
plot_uni(data, 'Employee Type', '% of accidents by Employee Type')
```



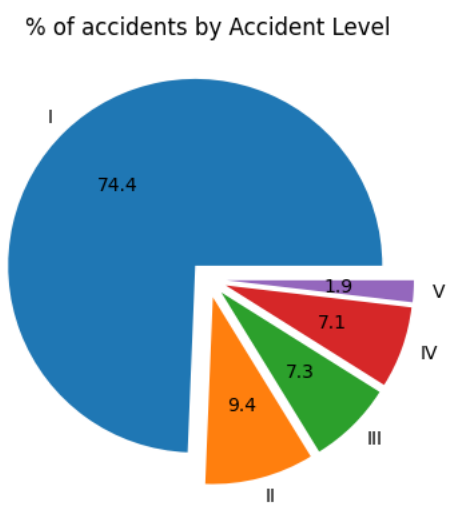
The minimum accidents are faced by the Third Party Remote workers as they are not always working on the sites. Overall 58% of the accidents are encountered Third Party employees.

```
plt.figure(figsize=(10, 5))
sns.countplot(data, x='Critical Risk')
plt.xticks(rotation=90)
plt.show()
```

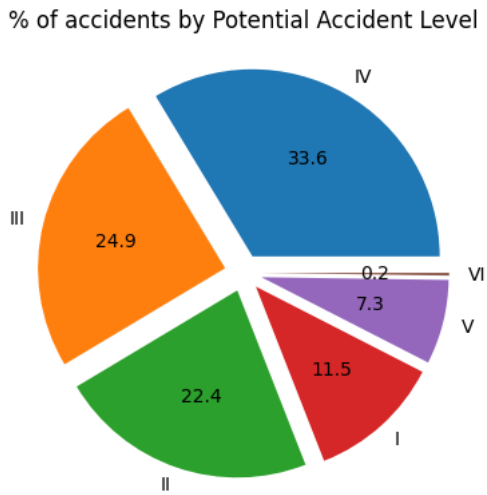


Most of the risks are grouped under Others which is about 50% of the dataset. The more common risks observed in the accidents are: Pressed, Manual Tools, Chemical Substances, Venomous Animals, Projection & Cut.

```
plot_uni(data, 'Accident Level', '% of accidents by Accident Level')
```




```
plot_uni(data, 'Potential Accident Level', '% of accidents by Potential Accident Level')
```



Observations:

1. Accident Level: Majority of the accidents fall under Level I occuring due to minor misses, while Level II, III & IV have similar frequency of occurence. High risk or severe accidents under Level V are only 2%.
2. Potential Accident Level: 81% accidents could potentially be moderately severe onces. and only 7.5% could've potentially been the most severe ones.
3. Potential Accident Level differs from the Accident Level. Majority are Level I accidents wheareas potential accidents have a majority in moderately severe ones (Level II, III & IV together).
4. Potential Accident Level: It has 6 Accident levels where the Level VI has only 1 accident recorded. To simplify and be able to compare the Accident & Potential Accident Level, we can change this record to the closest level V.

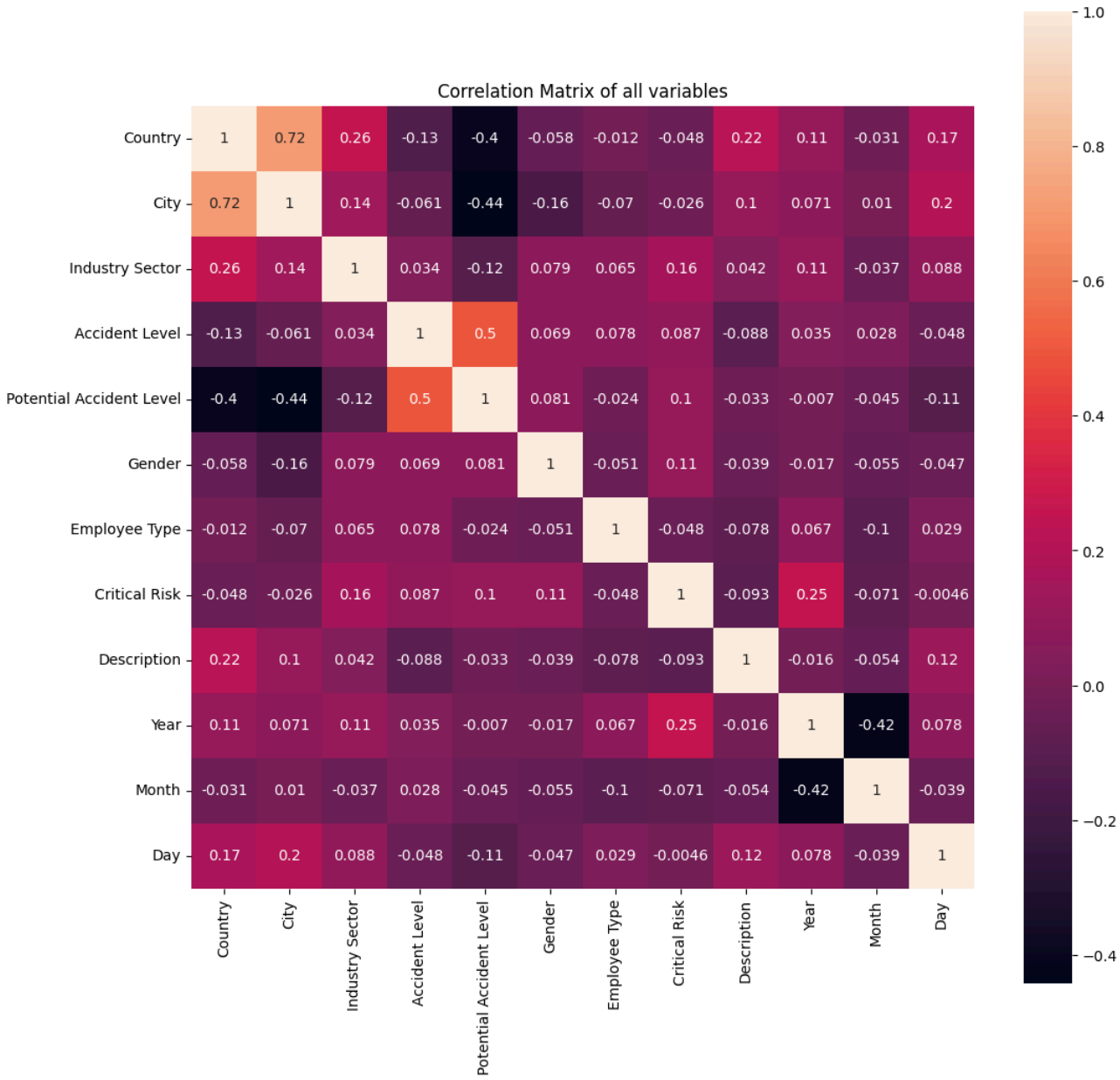
```
# Changing Accident Level VI to closest Level V
data['Potential Accident Level'].replace('VI', 'V', inplace=True)
data[data['Potential Accident Level']=='VI']
```

Date	Country	City	Industry Sector	Accident Level	Potential Accident Level	Gender	Employee Type	Critical Risk	Description	Year	Month	Day

MULTIVARIATE ANALYSIS

```
labenc = LabelEncoder()
df = data.iloc[:,1:].apply(labenc.fit_transform)

plt.figure(figsize=(12, 12))
plt.title('Correlation Matrix of all variables')
sns.heatmap(df.corr(), annot=True, square=True)
```

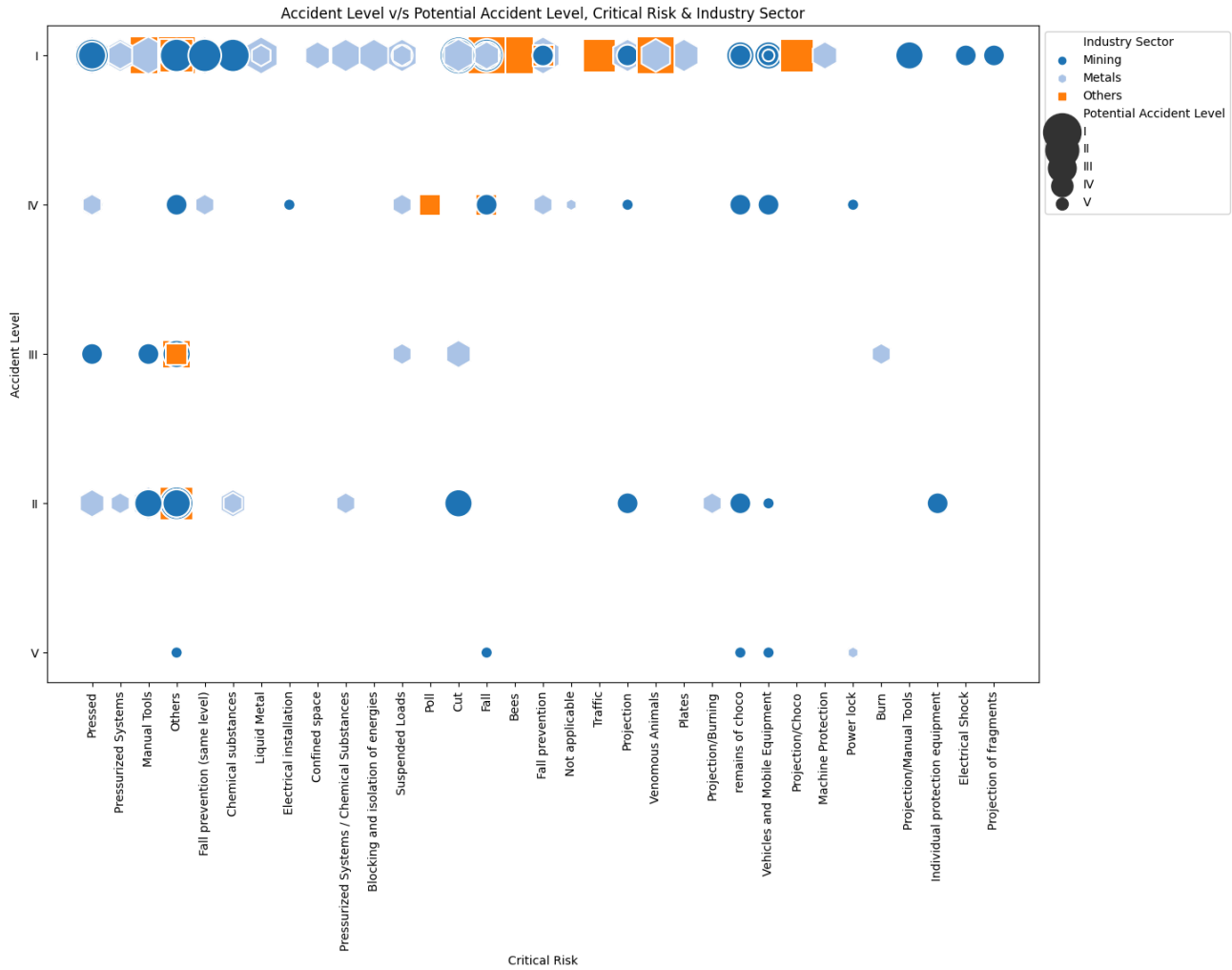
```
<Axes: title={'center': 'Correlation Matrix of all variables'}>
```



Observations:

1. The highest correlation is seen between City & Country, which is also the most obvious expectation.
2. Accident Level & Potential Accident Level have only 50% correlation.
3. There is only low to moderate correlation among the other variables.

```
fig, axes = plt.subplots(figsize=(15, 10))
marker_styles = {'Mining':'o', 'Metals':'h', 'Others':'s'}
size_order = list(data['Potential Accident Level'].sort_values(ascending=True).unique())
sns.scatterplot(data=data, y='Accident Level', x='Critical Risk', hue='Industry Sector',
                style='Industry Sector', markers=marker_styles,
                size='Potential Accident Level', size_order=size_order, ax=axes, sizes=(100, 1000), palette='tab20')
axes.set_title('Accident Level v/s Potential Accident Level, Critical Risk & Industry Sector')
axes.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.xticks(rotation=90)
plt.show()
```



Observations:

1. Most of the accidents occured are the least severe ones (level I), and majority of them could've potentially been more severe as Levels II & III accidents.
2. The most common risk seen in all industries is related to Fall - Fall & Fall Prevention.
3. All the most severe accidents (level V) are also potentially the most severe accidents, and all the risks associated are seen in Mining industry, except 'Power Lock' which is observed as a critical risk for these accidents in Metals industry.
4. Some of the level I accidents in Mining industry could have been level IV accidents potentially, with different risks like Electrical Installation, Projection & Power Lock.
5. All the Level IV accidents are also marked as potentially Level IV accidents only, except accidents caused to risks of Elecrtrical Installation, Projection, Burn & one accident with no risks (Not Applicable).
6. The following risks are only seen in Metals industry: Pressurized Systems, Liquid Metal, Confined Space, Chemicals Substances, Blocking and isolation of engines, Suspended Loads, Burn, Machine Protection, with majority of them observed in Level I accidents.
7. Majority of Level II accidents could've potentially been Level III & IV accidents, except the ones with the risks associated in the 'Others' category.

```
def plot_bi(df, col, huecol, title, ax):
    ax = sns.countplot(data, x=col, order=data[col].value_counts().index, hue=huecol, ax=ax)
```

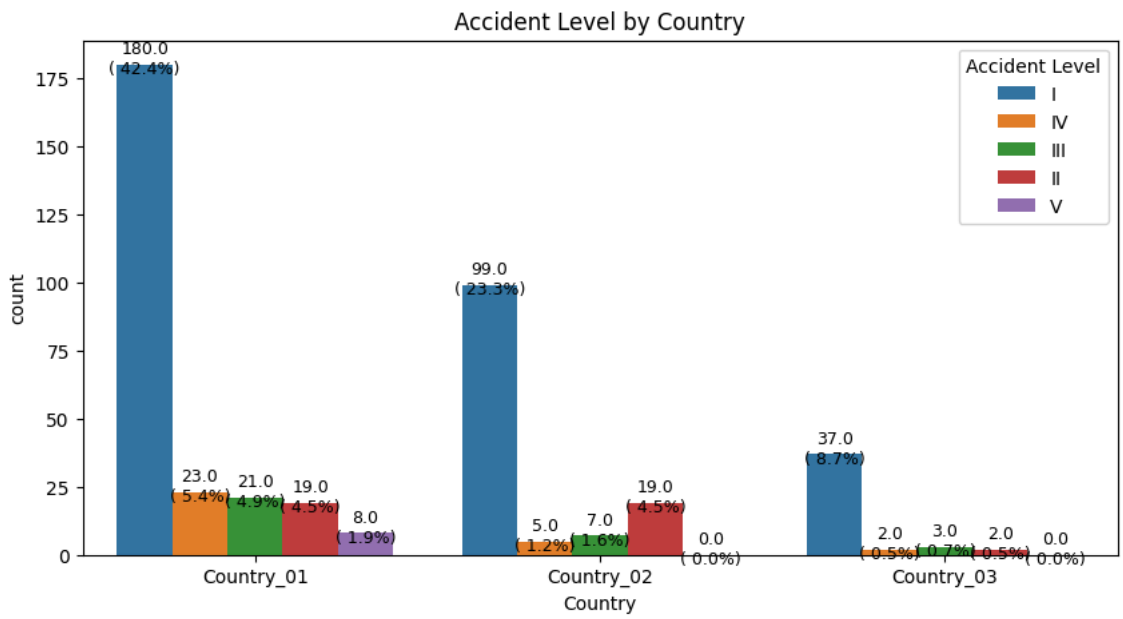
```
    total = len(data[col])
    for p in ax.patches:
        ht = p.get_height()
        ax.annotate(f'{ht}\n({ht / total: .1%})',
                    (p.get_x() + p.get_width() / 2., ht), ha='center', va='center',
                    size=9, xytext=(0, 3), textcoords='offset points')
    ax.set_title(title)
    return ax
```

Understanding Country-City mapping first

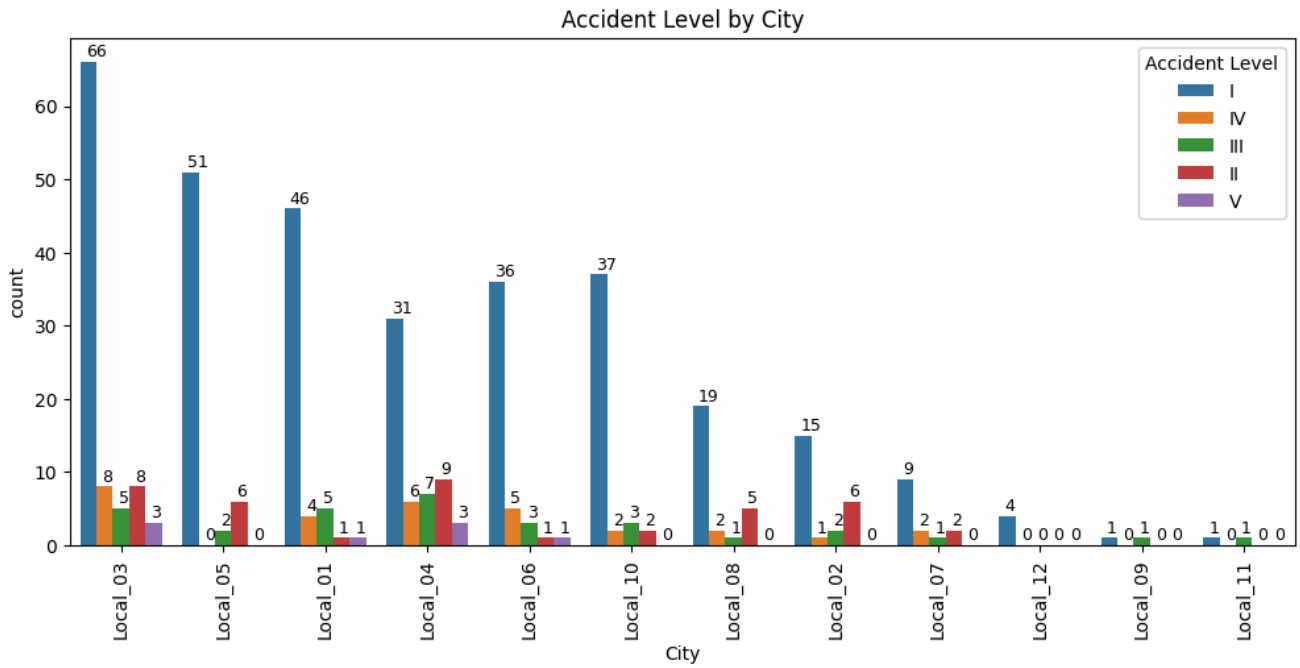
```
data[['Country', 'City']].drop_duplicates().sort_values(['Country', 'City']).reset_index(drop=True)
```

	Country	City
0	Country_01	Local_01
1	Country_01	Local_03
2	Country_01	Local_04
3	Country_01	Local_06
4	Country_01	Local_11
5	Country_02	Local_02
6	Country_02	Local_05
7	Country_02	Local_07
8	Country_02	Local_08
9	Country_02	Local_09
10	Country_02	Local_12
11	Country_03	Local_10

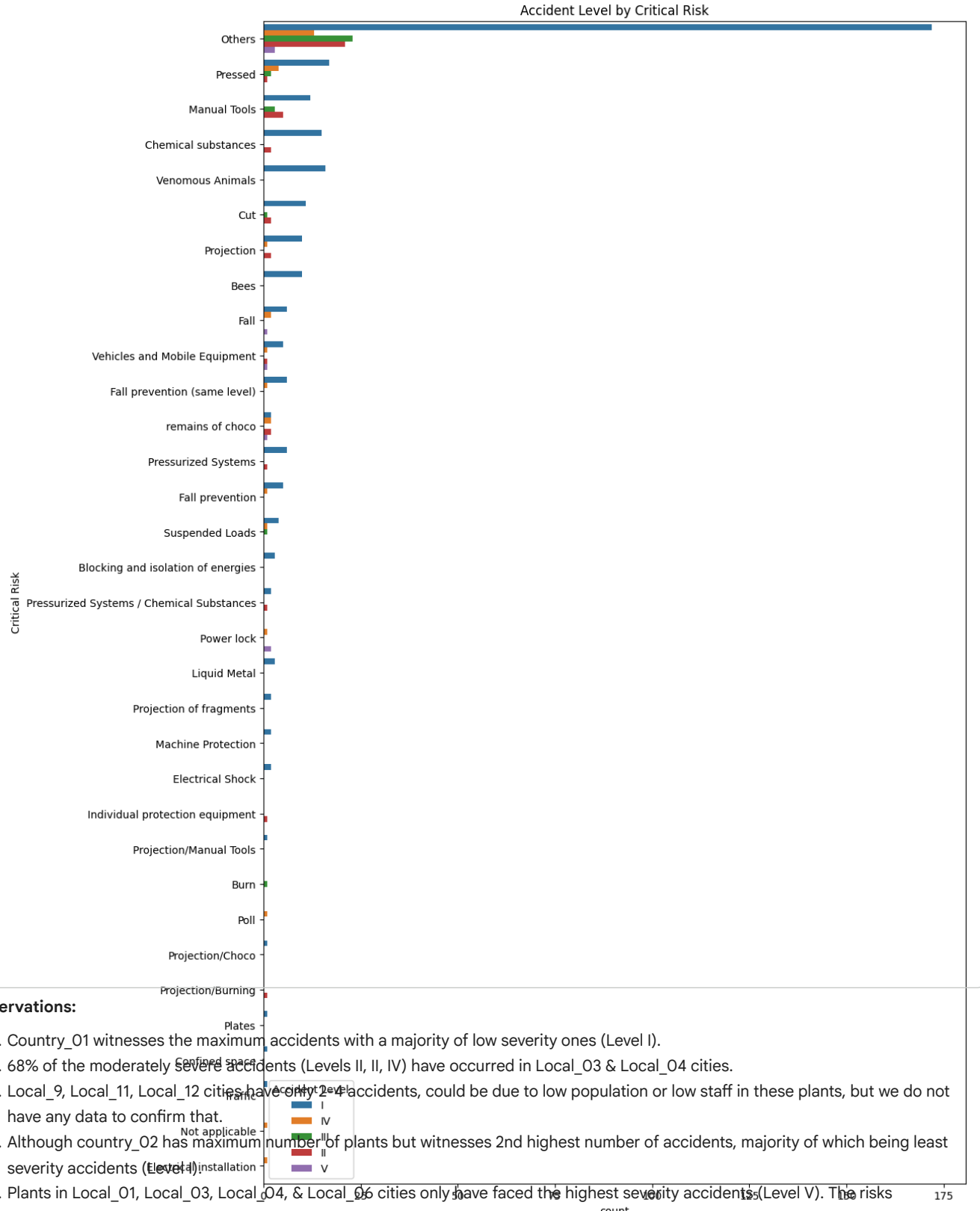
```
figure, axes = plt.subplots(1, 1, figsize=(10, 5))
plot_bi(data, 'Country', 'Accident Level', 'Accident Level by Country', axes)
plt.show()
```




```
plt.figure(figsize=(12, 5))
ax = sns.countplot(data, x='City', order=data['City'].value_counts().index, hue='Accident Level')
ax.set_xticklabels(ax.get_xticklabels(), rotation=90)
for p in ax.patches:
    ax.annotate('{:.0f}'.format(p.get_height()),
                (p.get_x(), p.get_height()), size=9, xytext=(3, 3), textcoords='offset points')
ax.set_title('Accident Level by City')
plt.show()
```



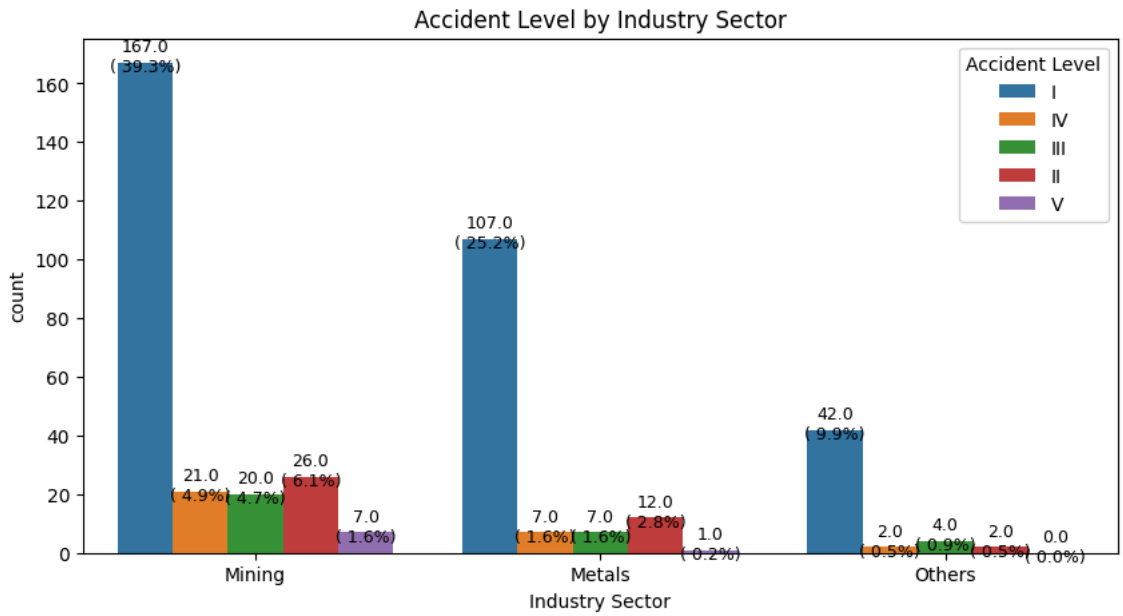

```
plt.figure(figsize=(12, 20))
sns.countplot(data, y='Critical Risk', order=data['Critical Risk'].value_counts().index, hue='Accident Level')
plt.title('Accident Level by Critical Risk')
plt.show()
```



Observations:

1. Country_01 witnesses the maximum accidents with a majority of low severity ones (Level I).
2. 68% of the moderately severe accidents (Levels II, II, IV) have occurred in Local_03 & Local_04 cities.
3. Local_9, Local_11, Local_12 cities have only 2-4 accidents, could be due to low population or low staff in these plants, but we do not have any data to confirm that.
4. Although country_02 has maximum number of plants but witnesses 2nd highest number of accidents, majority of which being least severity accidents (Level I).
5. Plants in Local_01, Local_03, Local_04, & Local_06 cities only have faced the highest severity accidents (Level V). The risks associated could be Fall, Vehicles & Mobile Equipment, Remains of Choco & Power Lock and some are categorized under 'Others'.
6. Most of the risks have contributed to least severe accidents of Level I, majority of which are recorded in Country_01.
7. Risks like Cut, Chemical Substances, Manual Tools, Pressed, Projection are the causes of moderate severity accidents (Level II, II & IV)

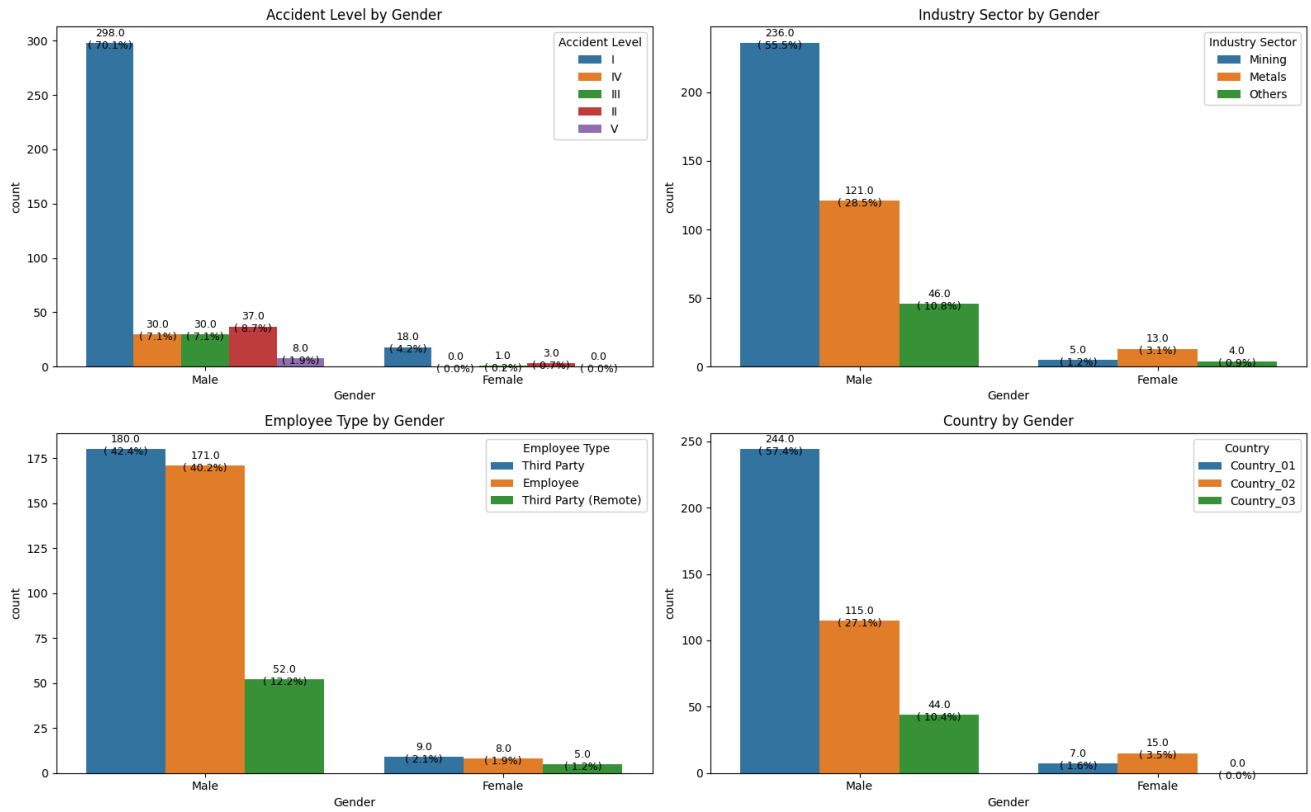
```
figure, axes = plt.subplots(1, 1, figsize=(10, 5))
plot_bi(data, 'Industry Sector', 'Accident Level', 'Accident Level by Industry Sector', axes)
plt.show()
```



Observations:

1. The most severe accidents (level V) are seen only in Mining & Metals industry sectors, where only 1 has occured in Metals industry & the rest in Mining industry.
2. The 'Others' industry sector has least accidents, with majority being the least severe ones (level I).
3. Accident Levels III & IV are almost equally distribute in Mining & Metals industry, while Level II accidents have occured more in both.

```
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

plot_bi(data, 'Gender', 'Accident Level', 'Accident Level by Gender', axes[0, 0])
plot_bi(data, 'Gender', 'Industry Sector', 'Industry Sector by Gender', axes[0, 1])
plot_bi(data, 'Gender', 'Employee Type', 'Employee Type by Gender', axes[1, 0])
plot_bi(data, 'Gender', 'Country', 'Country by Gender', axes[1, 1])

plt.tight_layout()
plt.show()
```

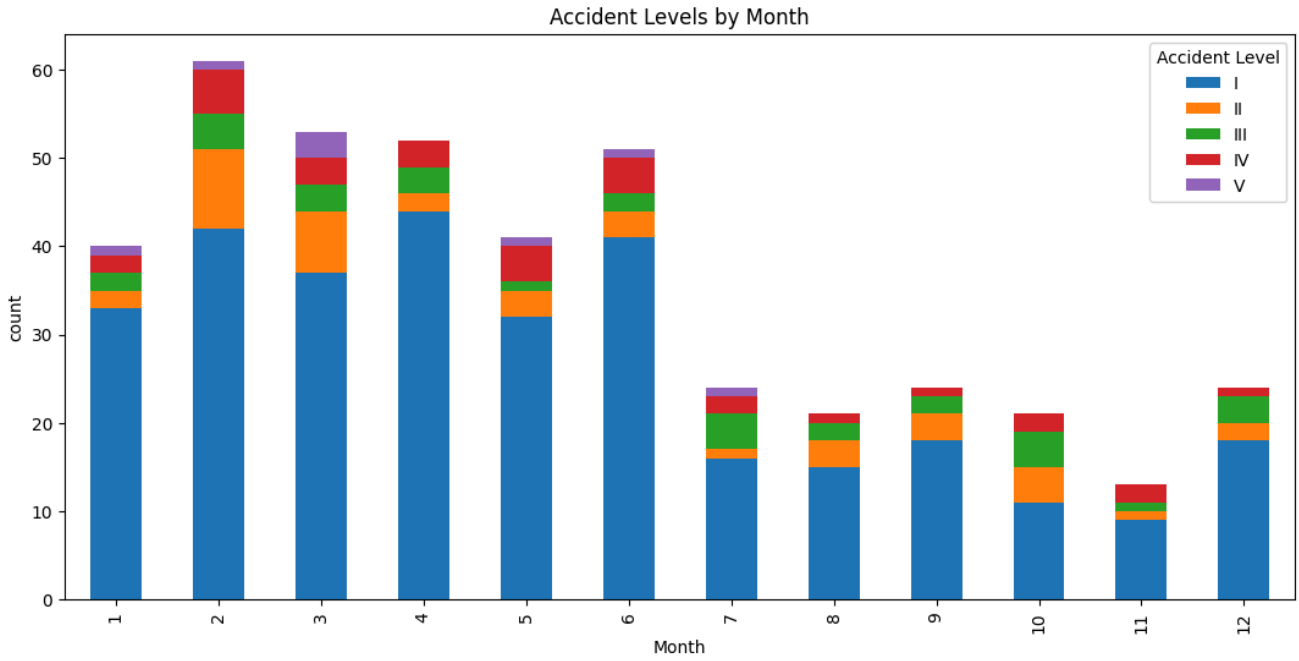


Observations:

1. There are no females suffered from an accident in Country_03, but even in other 2 countries together, only 6% of the accidents have females involved.
2. About 60% of the females met with accident in Metals industry, and 23% in Mining industry, while the rest in 'Others'.
3. There is almost equal distribution of females as different types of employees.
4. 82% of the females were involved in least severity accidents (Level I) and the rest in moderate severity ones (levels II, III, IV).
5. 87% of the men, with almost equal distribution, are either direct Employees or Third Party employees.
6. As the accident of level I have occured more, 74% of men have met with them, but only 2% of men have suffered from most severe accidents (Level V), with no women involved in these.

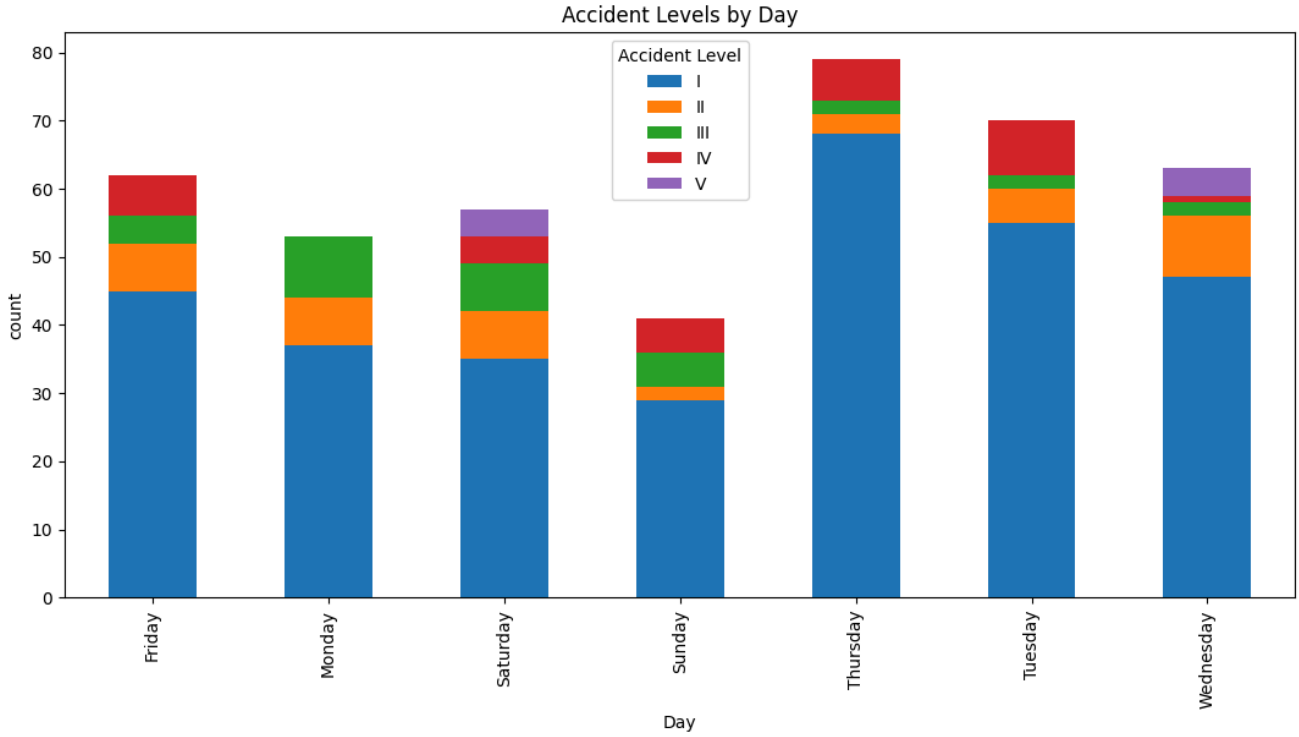
```
count = pd.crosstab(columns=data['Accident Level'], index=data['Month'])
ax = count.plot(kind='bar', stacked=True, figsize=(13, 6))
ax.set_title('Accident Levels by Month')
ax.set_ylabel('count')
```

```
Text(0, 0.5, 'count')
```




```
count = pd.crosstab(columns=data['Accident Level'], index=data['Day'])
ax = count.plot(kind='bar', stacked=True, figsize=(13, 6))
ax.set_title('Accident Levels by Day')
ax.set_ylabel('count')
```

```
Text(0, 0.5, 'count')
```



Observations:

1. Majority of potentially high severity accidents (level V) are seen in the month of March and especially on Wednesdays & Saturdays of the week.

2. Majority of moderate severity accidents (levels II, III, IV) are seen in February & March.
3. Majority of potentially least severe accidents (level I) are seen in first half of the year. But these numbers don't include data post 9th July 2017, hence this does not provide a full view.
4. Monday has witnessed only Level I, II & II accidents, while Level IV accidents have mostly occured on Tuesday, Thursday, Friday & Sunday.

Step 3: Data preprocessing (NLP Preprocessing techniques)

We will work on the Description column here, which will help us identify the real causes of these accidents.

```
pp_data = data.copy()
pp_data['Description'].head()
```

```
0                                                                                                  While removing
the drill rod of the Jumbo 08 for maintenance, the supervisor proceeds to loosen the support of the intermediate centralizer to
facilitate the removal, seeing this the mechanic supports one end on the drill of the equipment to pull with both hands the bar
and accelerate the removal from this, at this moment the bar slides from its point of support and tightens the fingers of the
mechanic between the drilling bar and the beam of the jumbo.
1
During the activation of a sodium sulphide pump, the piping was uncoupled and the sulfide solution was designed in the area to
reach the maid. Immediately she made use of the emergency shower and was directed to the ambulatory doctor and later to the
hospital. Note: of sulphide solution = 48 grams / liter.
2
In the sub-station MILPO located at level +170 when the collaborator was doing the excavation work with a pick (hand tool),
hitting a rock with the flat part of the beak, it bounces off hitting the steel tip of the safety shoe and then the metatarsal
area of the left foot of the collaborator causing the injury.
3    Being 9:45 am. approximately in the Nv. 1880 CX-695 OB7, the personnel begins the task of unlocking the Soquet bolts of
the BHB machine, when they were in the penultimate bolt they identified that the hexagonal head was worn, proceeding Mr.
Cristóbal - Auxiliary assistant to climb to the platform to exert pressure with your hand on the "DADO" key, to prevent it from
coming out of the bolt; in those moments two collaborators rotate with the lever in anti-clockwise direction, leaving the key
of the bolt, hitting the palm of the left hand, causing the injury.
4                                                                    Approximately at 11:45 a.m. in circumstances
that the mechanics Anthony (group leader), Eduardo and Eric Fernández-injured-the three of the Company IMPROMEC, performed the
removal of the pulley of the motor of the pump 3015 in the ZAF of Marcy. 27 cm / Length: 33 cm / Weight: 70 kg), as it was
locked proceed to heating the pulley to loosen it, it comes out and falls from a distance of 1.06 meters high and hits the
instep of the right foot of the worker, causing the injury described.
Name: Description, dtype: object
```

```
# Checking if there are any UTF-8 characters
def check_beyond_ascii(text):
    return all(ord(char) > 128 for char in text)

pp_data['Has_outside_ASCII'] = pp_data['Description'].apply(check_beyond_ascii)

if((pp_data['Has_outside_ASCII'].unique()[0]==False) | (len(pp_data['Has_outside_ASCII'].unique())==2)):
    print(pp_data['Has_outside_ASCII'].unique(), ': There are no characters beyond the range of ASCII character set.')
else:
    print(pp_data['Has_outside_ASCII'].unique(), ': There are characters beyond the range of ASCII character set. Please re
pp_data.drop('Has_outside_ASCII', axis=1, inplace=True)
```

```
[False] : There are no characters beyond the range of ASCII character set.
```

```
# Checking if there are any HTML tags or URLs
def check_html_urls(text):
    check_pattern = re.compile(r'https?://\S+|www\.\S+|<[^>]+>')
    return bool(check_pattern.search(text))

# Assuming pp_data is a DataFrame with a 'Description' column
pp_data['Has_HTML_URLs'] = pp_data['Description'].apply(check_html_urls)
if((pp_data['Has_HTML_URLs'].unique()[0]==False) | (len(pp_data['Has_HTML_URLs'].unique())==2)):
    print(pp_data['Has_HTML_URLs'].unique(), ': There are no HTML tags or URLs in the data.')
else:
    print(pp_data['Has_HTML_URLs'].unique(), ': There are HTML tags or URLs in the data. Please remove them.')
pp_data.drop('Has_HTML_URLs', axis=1, inplace=True)
```

```
[False] : There are no HTML tags or URLs in the data.
```

There are no characters beyond ASCII character set range and no HTML tags or URLs in the dataset.
The numbers in the text are used for measurements, time, etc. hence provide meaningful information, but will not be helpful during vectorization and modelling, as it would add more noise and sparsity. Hence, we decided to retain only alphabetic characters.

PRE-PROCESSING FUNCTION

```
def pre_processing(text):
    # Lowercasing the entire text
    text = text.lower()

    # Dealing with accented characters and unicodes by using Unicode normalization
    text = unidecode(text)
    text = unicodedata.normalize('NFKD', text)

    # Retaining only alphabetic characters
    text = ' '.join(re.compile(r'[a-z]+').findall(text))

    # Removing English Stopwords
    stopwords_list = set(stopwords.words('english'))
    text = ' '.join([word for word in word_tokenize(text) if word not in stopwords_list])

    # Lemmatizing the text now
    lemmatizer = WordNetLemmatizer()
    text = ' '.join([lemmatizer.lemmatize(word) for word in word_tokenize(text)])

    return text
```

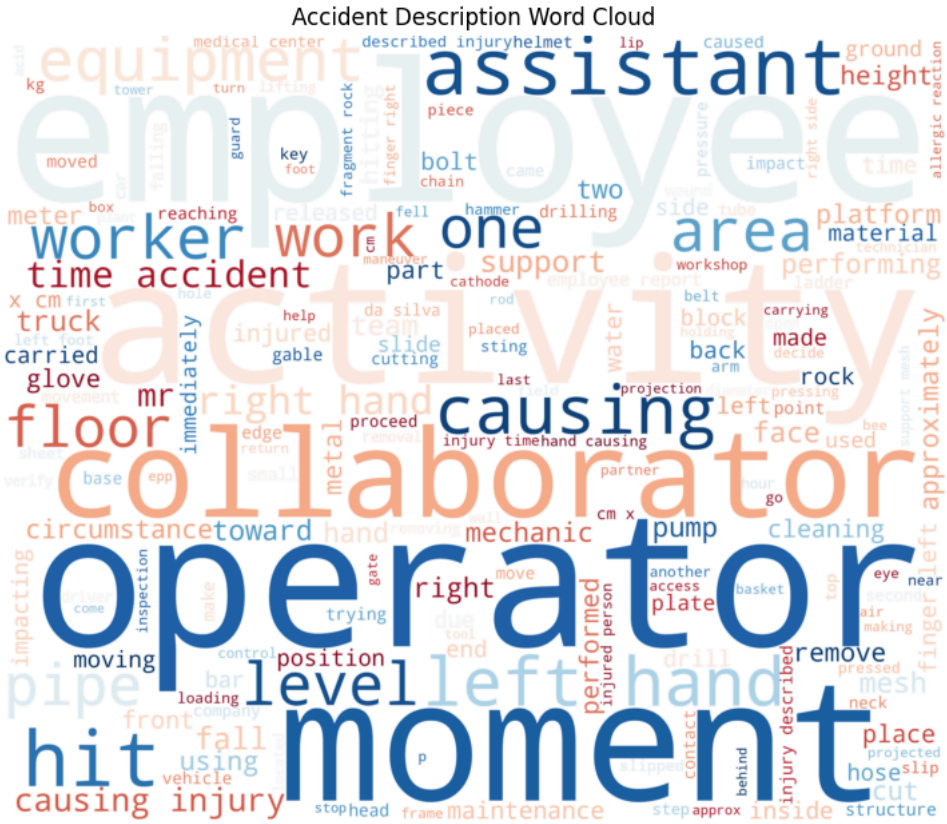
```
pp_data['Description'] = pp_data['Description'].apply(pre_processing)
```

```
pp_data['Description'].head()
```

```
0                                                                    removing drill rod jumbo maintenance supervisor
proceeds loosen support intermediate centralizer facilitate removal seeing mechanic support one end drill equipment pull hand
bar accelerate removal moment bar slide point support tightens finger mechanic drilling bar beam jumbo
1
activation sodium sulphide pump piping uncoupled sulfide solution designed area reach maid immediately made use emergency
shower directed ambulatory doctor later hospital note sulphide solution gram liter
2
sub station milpo located level collaborator excavation work pick hand tool hitting rock flat part beak bounce hitting steel
tip safety shoe metatarsal area left foot collaborator causing injury
3    approximately nv cx ob personnel begin task unlocking soquet bolt bhb machine penultimate bolt identified hexagonal head
worn proceeding mr cristobal auxiliary assistant climb platform exert pressure hand dado key prevent coming bolt moment two
collaborator rotate lever anti clockwise direction leaving key bolt hitting palm left hand causing injury
4                                                  approximately circumstance mechanic anthony group leader eduardo eric
fernandez injured three company impromec performed removal pulley motor pump zaf marcy cm length cm weight kg locked proceed
heating pulley loosen come fall distance meter high hit instep right foot worker causing injury described
Name: Description, dtype: object
```

```
descriptions = ' '.join(pp_data['Description'])
descriptions = ' '.join([word for word in descriptions.split()])

descriptions_wc = WordCloud(stopwords=STOPWORDS, colormap='RdBu', background_color='white',
                            width=3000, height=2500).generate(descriptions)

plt.figure(figsize=(20, 20))
plt.subplot(1, 2, 1)
plt.imshow(descriptions_wc, interpolation='bilinear')
plt.axis('off')
plt.title('Accident Description Word Cloud')
```

```
Text(0.5, 1.0, 'Accident Description Word Cloud')
```



Let's perform POS tagging to verify this.

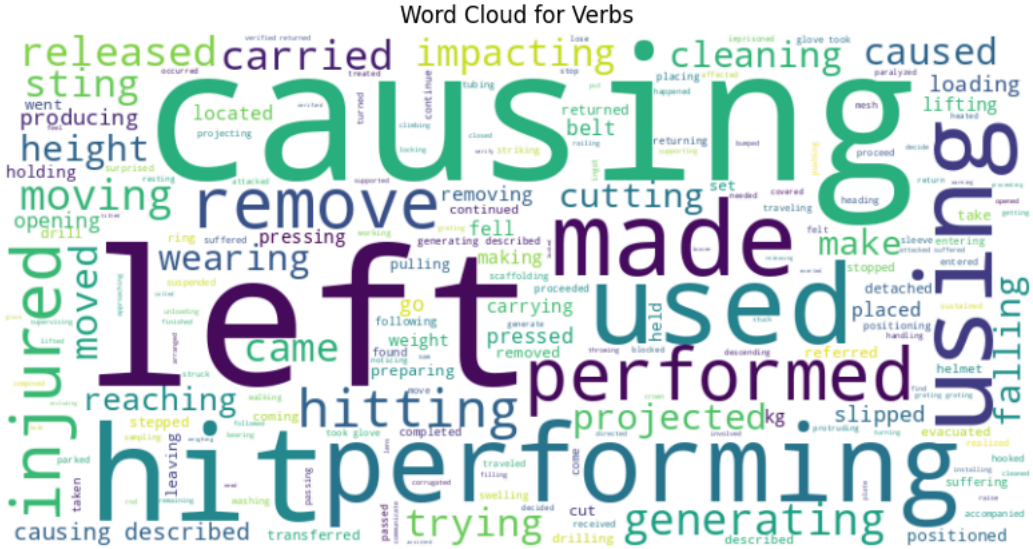
POS TAGGING

```
description_list = pp_data['Description'].tolist()
pos_tags = [pos_tag(word_tokenize(desc)) for desc in description_list]
pp_data['POS_Tags'] = pos_tags
```

```
# Flatten the list of POS tags
flat_pos_tags = [tag[0] for sublist in pos_tags for _, tag in sublist]

# Get all nouns and verbs
nouns = [word for word, tag in zip(word_tokenize(" ".join(description_list)), flat_pos_tags) if tag.startswith('N')]
verbs = [word for word, tag in zip(word_tokenize(" ".join(description_list)), flat_pos_tags) if tag.startswith('V')]

# Create WordCloud for Nouns
noun_wc = WordCloud(width=800, height=400, background_color='white').generate(" ".join(nouns))
plt.figure(figsize=(10, 5))
plt.imshow(noun_wc, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud for Nouns')
plt.show()

# Create WordCloud for Verbs
verb_wc = WordCloud(width=800, height=400, background_color='white').generate(" ".join(verbs))
plt.figure(figsize=(10, 5))
plt.imshow(verb_wc, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud for Verbs')
plt.show()
```

Word Cloud for Nouns

Word Cloud for Verbs



Observations:

1. There are a lot of action words like: falling, cutting, drilling, hitting, cleaning, slipped, injured, carried, moving, pressing, etc. These actions must have led to the accident.
2. There are a lot of nouns which can also help us understand the object causing the accident, like: truck, pipe, bee, vehicle, tube, floor, support mesh, ladder, helmet, metal, bolt, rock, water, tool, sting, equipment, etc.

```
pp_data.head()
```

	Date	Country	City	Industry Sector	Accident Level	Potential Accident Level	Gender	Employee Type	Critical Risk	Description	Year	Month	Day
0	2016-01-01	Country_01	Local_01	Mining	I	IV	Male	Third Party	Pressed	removing drill rod jumbo maintenance supervisor proceeds loosen support intermediate centralizer facilitate removal seeing mechanic support one end drill	2016	1	Friday

Step 4: Data preparation - Cleansed data in .xlsx or .csv file

TOKENIZATION

```
# Using SpaCy for Tokenization
spacy_eng = spacy.load('en_core_web_sm')
pp_data['Description'] = pp_data['Description'].astype(str)
pp_data['Tokenized Text'] = pp_data['Description'].apply(lambda x: [token.text for token in spacy_eng(x)])
pp_data.head()
```

	Date	Country	City	Industry Sector	Accident Level	Potential Accident Level	Gender	Employee Type	Critical Risk	Description	Year	Month	Day
0	2016-01-01	Country_01	Local_01	Mining	I	IV	Male	Third Party	Pressed	removing drill rod jumbo maintenance supervisor proceeds loosen support intermediate centralizer facilitate removal seeing mechanic support one end drill equipment pull hand bar accelerate removal moment bar slide point support	2016	1	Friday

N-GRAMS

```
# Finding n-grams for the Description
def generate_ngrams(text, n):
    doc = spacy_eng(text)
    ngrams = list(zip(*[doc[i:] for i in range (n)]))
    return [' '.join(tokens.text for tokens in ng) for ng in ngrams]

pp_data['Uni-grams'] = pp_data['Description'].apply(lambda x: generate_ngrams(x, 1))
pp_data['Bi-grams'] = pp_data['Description'].apply(lambda x: generate_ngrams(x, 2))
pp_data['Tri-grams'] = pp_data['Description'].apply(lambda x: generate_ngrams(x, 3))

pp_data.head()
```

	Date	Country	City	Industry Sector	Accident Level	Potential Accident Level	Gender	Employee Type	Critical Risk	Description	Year	Month	Day
0	2016-01-01	Country_01	Local_01	Mining	I	IV	Male	Third Party	Pressed	removing drill rod jumbo maintenance supervisor proceeds loosen support intermediate centralizer facilitate removal seeing mechanic support one end drill equipment pull hand bar accelerate removal moment bar slide point support tightens finger mechanic drilling bar beam jumbo	2016	1	Friday

```
# Let's find the most frequent n-grams
unigm = [ng for sublist in pp_data['Uni-grams'] for ng in sublist]
bigm = [ng for sublist in pp_data['Bi-grams'] for ng in sublist]
trigm = [ng for sublist in pp_data['Tri-grams'] for ng in sublist]

freq_unigm = pd.DataFrame(Counter(unigm).most_common(), columns=['Uni-gram', 'Frequency'])
freq_bigm = pd.DataFrame(Counter(bigm).most_common(), columns=['Bi-gram', 'Frequency'])
freq_trigm = pd.DataFrame(Counter(trigm).most_common(), columns=['Tri-gram', 'Frequency'])

print('Most frequent Uni-grams: \n', freq_unigm.head(20))
print('Most frequent Bi-grams: \n', freq_bigm.head(20))
print('Most frequent Tri-grams: \n', freq_trigm.head(20))
```

```
Most frequent Uni-grams:
         Uni-gram  Frequency
0       employee        183
1           hand        181
2        causing        166
3           left        160
4          right        155
5       operator        133
6       activity        120
7           time        112
8         injury        111
```

```
9         moment        103
10        worker         90
11  collaborator         84
12          area         81
13           one         80
14          work         80
15     equipment         78
16        finger         76
17     assistant         75
18      accident         73
19          pipe         72
Most frequent Bi-grams:
               Bi-gram  Frequency
0          left hand         70
1         right hand         57
2      time accident         56
3     causing injury         55
4        finger left         25
5               x cm         23
6    employee report         22
7   injury described         20
8               cm x         18
9           da silva         18
10    medical center         17
11  described injury         17
12         left foot         15
13      hand causing         15
14     fragment rock         15
15    injured person         15
16      finger right         14
17      support mesh         14
18       injury time         14
19        right side         13
Most frequent Tri-grams:
                         Tri-gram  Frequency
0              finger left hand         24
1                       cm x cm         16
2      causing injury described         15
3             finger right hand         14
4          injury time accident         13
5        time accident employee          9
6           hand causing injury          8
7   generating described injury          8
8               manoel da silva          8
9       described time accident          7
10            left hand causing          6
```

The most frequent Bi-grams provide useful information regarding the cause of accident, while the unigrams seem to get covered in bi-grams and the tri-grams do not seem to be adding any useful information on top of bi-grams. For Example: 'injury', 'causing injury' & 'causing injury described'. Hence, we can use the range of only 2 during vector encodings.

SAVING THE PRE-PROCESSED DATA FILE IN .CSV FORMAT

```
output_path = str(folder + 'IHMStefanini_industrial_safety_and_health_cleansed.csv')
pp_data.to_csv(output_path, index=False)
```

VECTOR ENCODING VIA TF-IDF

```
tfidf_vectorizer = TfidfVectorizer(max_features=1000, stop_words='english', ngram_range=(2, 2))
tfidf_matrix = tfidf_vectorizer.fit_transform(description_list)
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=tfidf_vectorizer.get_feature_names_out())

tfidf_df.head()
```

	access aripuana	access ladder	accident employee	accident operator	accident victim	accident worker	accompanied geologist	activity area	activity chuteo	activity employee	...	work area	work moment	wo normal
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0

```
tfidf_df.shape
```

```
(425, 1000)
```

```
# Display the most important features
tfidf_scores = tfidf_df.sum(axis=0)
tfidf_scores_df = pd.DataFrame({'Feature': tfidf_vectorizer.get_feature_names_out(), 'TF-IDF Score': tfidf_scores})

top_features = tfidf_scores_df.sort_values(by='TF-IDF Score', ascending=False)

print("Top Features:")
print(top_features.head(30))
```

```
Top Features:
                                  Feature  TF-IDF Score
left hand                       left hand     16.026339
right hand                     right hand     13.926010
causing injury             causing injury     13.242063
time accident               time accident     10.120484
employee report           employee report      8.380265
finger left                   finger left      7.739658
injury described         injury described      6.511859
medical center             medical center      6.181397
left foot                       left foot      5.524003
right leg                       right leg      5.479867
described injury         described injury      5.202557
finger right                 finger right      4.763424
hand causing                 hand causing      4.761339
cm cm                                cm cm      4.331312
generating injury       generating injury      4.217899
injured person             injured person      4.186464
hit right                       hit right      4.007624
fragment rock               fragment rock      3.973550
middle finger               middle finger      3.919644
circumstance worker  circumstance worker      3.892055
support mesh                 support mesh      3.778769
right foot                     right foot      3.681881
injury time                   injury time      3.656111
da silva                         da silva      3.262092
ring finger                   ring finger      3.258298
causing cut                   causing cut      3.255090
left leg                         left leg      3.049839
employee used               employee used      3.027345
accident employee      accident employee      2.984388
safety glove                 safety glove      2.938843
```

You can see most of the accidents are related to injuries on hands, fingers, feet, legs.

VECTOR ENCODING VIA WORD2VEC

```
tokens_list = pp_data['Tokenized Text'].to_list()
word2vec_vectorizer = Word2Vec(sentences=tokens_list, vector_size=1000, window=5, min_count=1, workers=4)

def get_sentence_vector(sentence, vectorizer):
    vector_sum = 0
    for word in sentence:
        if word in vectorizer.wv:
            vector_sum += vectorizer.wv[word]
    return vector_sum

pp_data['Description_Vector'] = [get_sentence_vector(sentence, word2vec_vectorizer) for sentence in tokens_list]

pp_data[['Description', 'Description_Vector']]
```

	Description	Description_Vector
0	removing drill rod jumbo maintenance supervisor proceeds loosen support intermediate centralizer facilitate removal seeing mechanic support one end drill equipment pull hand bar accelerate removal moment bar slide point support tightens finger mechanic drilling bar beam jumbo	[0.15689239, 0.032792345, 0.05961993, 0.05683695, -0.02947724, -0.0034363763, 0.031655412, 0.05739922, -0.03947713, 0.013594026, 0.05299458, 0.0085570505, -0.02622484, 0.017888717, 0.081248306, 0.005608923, -0.042488515, -0.0011145764, 0.06583888, -0.1323013, 0.038831003, -0.07133565, 0.009850043, 0.0008292134, 0.059876334, -0.048833333, 0.047934104, -0.042301547, -0.17362727, 0.06015557, 0.06294533, -0.057788353, -0.042411733, -0.04420411, 0.07734276, -0.040740024, 0.051727142, 0.020397183, -0.045424733, -0.11826102, -0.035560466, 0.035131626, -0.09915616, 0.11357415, 0.002158553, -0.016330179, -0.05025904, 0.07562341, -0.09700866, 0.022257652, -0.005843183, -0.024083812, -0.0019613118, -0.09930171, 0.006566635, -0.006647499, 0.051855892, -0.0111139575, -0.044542916, -0.03151444, -0.061752405, -0.016601332, -0.06650158, 0.031011915, -0.008954839, 0.08492317, 0.010387753, 0.1266273, -0.012362543, -0.016394136, 0.0044414583, -0.022411691, 0.061868895, -0.054563884, 0.008796975, 0.011809333, 0.017168403, -0.017170006, 0.017693201, -0.011211575, -0.029095253, -0.0046106465, -0.07204723, 0.10373141, -0.11738301, 0.018635124, -0.05141609, 0.046594393, 0.061971452, 0.058369733, -0.008974617, -0.0011599769, -0.018503767, 0.104227014, 0.100743085, 0.040123593, 0.111102946, -0.10119708, 0.043913122, 0.0351402, ...]
1	activation sodium sulphide pump piping uncoupled sulfide solution designed area reach maid immediately made use emergency shower directed ambulatory doctor later hospital note sulphide solution gram liter	[0.050151583, 0.0034160542, 0.013888093, 0.019545827, -0.009495543, -0.0049411515, 0.0074851587, 0.008454382, -0.014982616, -0.0011812993, 0.02055987, 0.0012473578, -0.0019640804, 0.0027686504, 0.025604615, 0.004107888, -0.011344254, 0.009619034, 0.018961314, -0.03065217, 0.009628636, -0.018888893, 0.0002450282, -0.0015737659, 0.01836113, -0.018088235, 0.011533625, -0.012984545, -0.047817048, 0.023033097, 0.015776353, -0.016042031, -0.008456894, -0.008074141, 0.02321849, -0.014243003, 0.013877783, 0.0060789613, -0.013908266, -0.03188104, -0.009820235, 0.007545152, -0.027684338, 0.02829795, -0.0070052794, -0.0020133932, -0.0137089025, 0.024054594, -0.024998188, 0.0014925421, 0.0025736364, -0.0053892313, 0.00097123, -0.026211115, -0.00318513, -0.002299205, 0.020697143, -0.0059520057, -0.008928609, -0.006984792, -0.014602349, -0.00635211, -0.017484184, 0.0071876277, -0.00022045439, 0.025726957, 0.004969235, 0.036033228, 0.0011531136, 0.0054210164, 0.0015911206, -0.010085412, 0.012869027, -0.014049785, 0.0063723796, -0.0038102316, 0.0078188535, 0.0010581037, 0.0027683747, -0.0007126773, -0.007915773, 0.0025455868, -0.021293368, 0.02934481, -0.029229503, 0.0004383106, -0.019439057, 0.014375143, 0.012542413, 0.019080745, 0.0008655561, -0.004556215, -0.009987159, 0.029731335, 0.021483961, 0.009396327, 0.035468724, -0.024848467, 0.008871979, 0.006677427, ...]
2	sub station milpo located level collaborator excavation work pick hand tool hitting rock flat part beak bounce hitting steel tip safety shoe metatarsal area left foot collaborator causing injury	[0.13369536, 0.021775207, 0.044225287, 0.04845478, -0.026594846, -0.006400733, 0.024580218, 0.05254709, -0.032500103, 0.0032595312, 0.037739847, 0.0062906463, -0.019858705, 0.021819685, 0.06587136, 0.0008959415, -0.03224192, 0.004564384, 0.057766948, -0.102979854, 0.032775044, -0.06029603, 0.005582173, -0.011247218, 0.052517686, -0.041682847, 0.04246109, -0.033545326, -0.14796437, 0.047016308, 0.05349432, -0.049784917, -0.03551207, -0.0338034, 0.05835721, -0.032338303, 0.040323827, 0.015126488, -0.039125368, -0.097195685, -0.02578402, 0.02642248, -0.07394707, 0.09401814, -0.0075016967, -0.016469533, -0.0324492, 0.061180234, -0.072055236, 0.016192403, 0.0066704014, -0.01868933, -0.006068812, -0.0752198, 0.0045858915, -0.0037188192, 0.045627937, -0.008454769, -0.037709557, -0.027312052, -0.046664804, -0.0056653405, -0.05444439, 0.018551145, -0.012987952, 0.06643607, 0.00820414, 0.09899785, -0.004545237, -0.010488166, 0.009071241, -0.015439571, 0.053027034, -0.044820193, 0.009297935, 0.010317022, 0.017584743, -0.0029092915, 0.0068162084, -0.008246895, -0.021885267, -0.0042402036, -0.05299078, 0.08486652, -0.09698965, 0.01767318, -0.031464268, 0.035150714, 0.05024454, 0.047147587, -0.008157161, -0.0022067332, -0.018508814, 0.082980245, 0.0812348, 0.023956954, 0.08450101, -0.08289387, 0.03084739, 0.02848848, ...]
3	approximately nv cx ob personnel begin task unlocking soquet bolt bhb machine penultimate bolt identified hexagonal head worn proceeding mr cristobal auxiliary assistant climb platform exert pressure hand dado key prevent coming bolt moment two collaborator rotate level anti clockwise direction leaving key bolt hitting palm left hand causing	[0.18112518, 0.031132653, 0.06670623, 0.06621624, -0.03092567, -0.013292035, 0.029057346, 0.063210405, -0.047927745, 0.0038906448, 0.06529022, 0.009712215, -0.03480629, 0.025089744, 0.09321976, 0.01323958, -0.04075098, 0.0068051126, 0.082492806, -0.14716396, 0.041240457, -0.08953734, 0.01655821, -0.013217548, 0.06721146, -0.05156313, 0.056454673, -0.044111714, -0.19256541, 0.06274254, 0.07441603, -0.060384456, -0.04117946, -0.03889714, 0.09309238, -0.044146564, 0.05496983, 0.02862508, -0.045644704, -0.14181994, -0.040344983, 0.020564443, -0.1059324, 0.12718509, -0.004693762, -0.015036279, -0.043544702, 0.07670733, -0.097035915, 0.018026844, 0.0013160921, -0.018027244, 0.0014883274, -0.11480094, 0.002823372, 0.0019745864, 0.073191285, -0.018723909, -0.04381191, -0.039665837, -0.06980168, -0.010882228, -0.074755535, 0.042889647, 0.014437665, 0.09657094, 0.008357919, 0.13710146, 0.0008278432, -0.014371306, -0.0021580413, -0.0233928, 0.06782872, -0.054603916, 0.014676921, 0.0093055405, 0.023848163, 0.00013720756, 0.015078905, -0.010067613, -0.03156231, 0.0028306092, -0.07440195, 0.116318025,

Step 5: Design, train and test basic machine learning classifiers

```
# Assigning the target vairable for modelling
Y = pp_data['Accident Level']
```

MODELLING USING TF-IDF VECTORS

```
# Splitting Training & Testing data
X_tf = tfidf_matrix

x_train_tf, x_test_tf, y_train, y_test = train_test_split(X_tf, Y, test_size=0.3, random_state=42, stratify=Y)
```

```
# Creating function for 5 different ML classifiers
def ml_classfiers(x_train, x_test, y_train, y_test):
    classifiers_dict = {
        'Logistic Regression': LogisticRegression(random_state=42),
        'Random Forest': RandomForestClassifier(n_estimators=100, max_depth=6),
        'K-Nearest Neighbor': KNeighborsClassifier(n_neighbors=50),
        'Support Vector Machine': SVC(kernel='rbf', C=1.0),
        'AdaBoost': AdaBoostClassifier(n_estimators=50, random_state=42)
```

```
    }

    classfier_list = []
    accuracy_scores = []
    precision_scores = []
    recall_scores = []
    f1_scores = []

    # Evaluate Model Performance
    for name, classifer in classifiers_dict.items():
        clf = classifer.fit(x_train, y_train)
        y_pred = classifer.predict(x_test)
        accuracy = float(accuracy_score(y_test, y_pred)) * 100
        report = classification_report(y_test, y_pred)
        conf_mx = confusion_matrix(y_test, y_pred)
        precision = float(report.split('\n')[-2].split()[2]) * 100
        recall = float(report.split('\n')[-2].split()[3]) * 100
        f1_score = float(report.split('\n')[-2].split()[4]) * 100
        classfier_list.append(name)
        accuracy_scores.append(accuracy)
        precision_scores.append(precision)
        recall_scores.append(recall)
        f1_scores.append(f1_score)
        print(name, ': \n')
        print('----------------------------------------------------------------')
        print('Classification Report: \n', report)
        print('Confusion Matrix: \n', conf_mx)

    # Create Performance Dataframe
    classifiers_df = pd.DataFrame({'Name':classfier_list, 'Accuracy (in %)': accuracy_scores,
                                   'Precision (in %)': precision_scores, 'Recall (in %)': recall_scores,
                                   'F1-Score (in %)': f1_scores})
    return classifiers_df
```

```
ml_classfiers_results_tfidf = ml_classfiers(x_train_tf, x_test_tf, y_train, y_test)
```

```
Logistic Regression :

----------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
         III       0.00      0.00      0.00         9
          IV       0.00      0.00      0.00         9
           V       0.00      0.00      0.00         3

    accuracy                           0.74       128
   macro avg       0.15      0.20      0.17       128
weighted avg       0.55      0.74      0.63       128

Confusion Matrix:
 [[95  0  0  0  0]
 [12  0  0  0  0]
 [ 9  0  0  0  0]
 [ 9  0  0  0  0]
 [ 3  0  0  0  0]]
Random Forest :

----------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
         III       0.00      0.00      0.00         9
          IV       0.00      0.00      0.00         9
           V       0.00      0.00      0.00         3

    accuracy                           0.74       128
   macro avg       0.15      0.20      0.17       128
weighted avg       0.55      0.74      0.63       128

Confusion Matrix:
 [[95  0  0  0  0]
 [12  0  0  0  0]
 [ 9  0  0  0  0]
 [ 9  0  0  0  0]
 [ 3  0  0  0  0]]
```

```
K-Nearest Neighbor :

--------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
         III       0.00      0.00      0.00         9
          IV       0.00      0.00      0.00         9
           V       0.00      0.00      0.00         3

    accuracy                           0.74       128
```

```
print('For models built on TF-IDF vectors:')
ml_classfiers_results_tfidf
```

For models built on TF-IDF vectors:

	Name	Accuracy (in %)	Precision (in %)	Recall (in %)	F1-Score (in %)
0	Logistic Regression	74.21875	55.0	74.0	63.0
1	Random Forest	74.21875	55.0	74.0	63.0
2	K-Nearest Neighbor	74.21875	55.0	74.0	63.0
3	Support Vector Machine	74.21875	55.0	74.0	63.0
4	AdaBoost	71.87500	55.0	72.0	62.0

Observations:

1. All the models, excpet AdaBoost, are providing the same accuracy of 74%, ~2% higher than AdaBoost. But for such a highly imbalanced dataset, it's better to use weighted average of Precision, Recall & F1-Score as performance metrics.
2. All the models provide very low performance, with only 55% Precision & 62-63% F1-Score, as only accidents of Level I have been correctly predicted and all the accidents in Level II - V are incorrectly predicted as Level I.
3. This is due to the dataset being small and highly imbalanced with majority of the accidents from Level I.

MODELLING USING WORD2VEC VECTORS

```
# Splitting Training & Testing data
X_wv = pp_data['Description_Vector'].to_list()

x_train_wv, x_test_wv, y_train, y_test = train_test_split(X_wv, Y, test_size=0.3, random_state=42, stratify=Y)
```

```
ml_classfiers_results_wv = ml_classfiers(x_train_wv, x_test_wv, y_train, y_test)
```

```
Logistic Regression :

--------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
         III       0.00      0.00      0.00         9
          IV       0.00      0.00      0.00         9
           V       0.00      0.00      0.00         3

    accuracy                           0.74       128
   macro avg       0.15      0.20      0.17       128
weighted avg       0.55      0.74      0.63       128

Confusion Matrix:
 [[95  0  0  0  0]
 [12  0  0  0  0]
 [ 9  0  0  0  0]
 [ 9  0  0  0  0]
 [ 3  0  0  0  0]]
Random Forest :

--------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
```

```
          III       0.00      0.00      0.00         9
           IV       0.00      0.00      0.00         9
            V       0.00      0.00      0.00         3

    accuracy                           0.74       128
   macro avg       0.15      0.20      0.17       128
weighted avg       0.55      0.74      0.63       128

Confusion Matrix:
 [[95  0  0  0  0]
 [12  0  0  0  0]
 [ 9  0  0  0  0]
 [ 9  0  0  0  0]
 [ 3  0  0  0  0]]
K-Nearest Neighbor :

----------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
         III       0.00      0.00      0.00         9
          IV       0.00      0.00      0.00         9
           V       0.00      0.00      0.00         3

    accuracy                           0.74       128
```

```
print('For models built on Word2Vec vectors:')
ml_classfiers_results_wv
```

For models built on Word2Vec vectors:

	Name	Accuracy (in %)	Precision (in %)	Recall (in %)	F1-Score (in %)
0	Logistic Regression	74.21875	55.0	74.0	63.0
1	Random Forest	74.21875	55.0	74.0	63.0
2	K-Nearest Neighbor	74.21875	55.0	74.0	63.0
3	Support Vector Machine	74.21875	55.0	74.0	63.0
4	AdaBoost	71.87500	56.0	72.0	63.0

You can observe that there is no difference in the model performance on both the Vectorization methods: TF-IDF & Word2Vec. Hence, going forward we can choose either one, we're selecting TF-TDF here as the importance of the words in a sentence matters to understand the cause of the accident and identify the Accident Level.

MODEL PERFORMANCE TUNING USING GRID SEARCH

```
# Creating function for 5 different ML classifiers
def ml_classfiers_with_grid_search(x_train, x_test, y_train, y_test):
    classifiers_dict = {
        'Logistic Regression': LogisticRegression(random_state=42),
        'Random Forest': RandomForestClassifier(),
        'K-Nearest Neighbor': KNeighborsClassifier(),
        'Support Vector Machine': SVC(kernel='rbf'),
        'AdaBoost': AdaBoostClassifier(random_state=42)
    }

    param_grid = {
        'Logistic Regression': {'C': [0.001, 0.01, 0.1, 1, 10, 100]},
        'Random Forest': {'n_estimators': [50, 100, 150], 'max_depth': [5, 10, 15]},
        'K-Nearest Neighbor': {'n_neighbors': [5, 10, 15, 20, 25]},
        'Support Vector Machine': {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto']},
        'AdaBoost': {'n_estimators': [20, 50, 100], 'learning_rate': [0.01, 0.1, 1]}
    }

    classfier_list = []
    accuracy_scores = []
    precision_scores = []
    recall_scores = []
    f1_scores = []

    for name, classifer in classifiers_dict.items():
        # Perform Grid Search
        grid_search = GridSearchCV(classifer, param_grid[name], scoring='accuracy', cv=5)
```

```
        grid_search.fit(x_train, y_train)
        best_model = grid_search.best_estimator_

        # Evaluate the Best Model
        y_pred = best_model.predict(x_test)
        accuracy = float(accuracy_score(y_test, y_pred)) * 100
        report = classification_report(y_test, y_pred)
        conf_mx = confusion_matrix(y_test, y_pred)
        precision = float(report.split('\n')[-2].split()[2]) * 100
        recall = float(report.split('\n')[-2].split()[3]) * 100
        f1_score = float(report.split('\n')[-2].split()[4]) * 100
        classfier_list.append(name)
        accuracy_scores.append(accuracy)
        precision_scores.append(precision)
        recall_scores.append(recall)
        f1_scores.append(f1_score)
        print(name, ': \n')
        print('----------------------------------------------------------------')
        print('Classification Report: \n', report)
        print('Confusion Matrix: \n', conf_mx)

    # Create Performance Dataframe
    classifiers_df = pd.DataFrame({'Name':classfier_list, 'Accuracy (in %)': accuracy_scores,
                                   'Precision (in %)': precision_scores, 'Recall (in %)': recall_scores,
                                   'F1-Score (in %)': f1_scores})
    return classifiers_df
```

```
ml_classfiers_results_tfidf_gs = ml_classfiers_with_grid_search(x_train_tf, x_test_tf, y_train, y_test)
```

```
Logistic Regression :

----------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
         III       0.00      0.00      0.00         9
          IV       0.00      0.00      0.00         9
           V       0.00      0.00      0.00         3

    accuracy                           0.74       128
   macro avg       0.15      0.20      0.17       128
weighted avg       0.55      0.74      0.63       128

Confusion Matrix:
 [[95  0  0  0  0]
 [12  0  0  0  0]
 [ 9  0  0  0  0]
 [ 9  0  0  0  0]
 [ 3  0  0  0  0]]
Random Forest :

----------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
         III       0.00      0.00      0.00         9
          IV       0.00      0.00      0.00         9
           V       0.00      0.00      0.00         3

    accuracy                           0.74       128
   macro avg       0.15      0.20      0.17       128
weighted avg       0.55      0.74      0.63       128

Confusion Matrix:
 [[95  0  0  0  0]
 [12  0  0  0  0]
 [ 9  0  0  0  0]
 [ 9  0  0  0  0]
 [ 3  0  0  0  0]]
K-Nearest Neighbor :

----------------------------------------------------------------
Classification Report:
               precision    recall  f1-score   support

           I       0.74      1.00      0.85        95
          II       0.00      0.00      0.00        12
```

```
          III       0.00      0.00      0.00         9
           IV       0.00      0.00      0.00         9
            V       0.00      0.00      0.00         3

     accuracy                           0.74       128
    macro avg       0.15      0.20      0.17       128
```

```
print('For models built on TF-IDF vectors and tuned using Grid Search:')
ml_classfiers_results_tfidf
```

```
For models built on TF-IDF vectors and tuned using Grid Search:
```

	Name	Accuracy (in %)	Precision (in %)	Recall (in %)	F1-Score (in %)
0	Logistic Regression	74.21875	55.0	74.0	63.0
1	Random Forest	74.21875	55.0	74.0	63.0
2	K-Nearest Neighbor	74.21875	55.0	74.0	63.0
3	Support Vector Machine	74.21875	55.0	74.0	63.0
4	AdaBoost	71.87500	55.0	72.0	62.0

You can observe that there is no difference in the model performance, even after using Grid Search method to fine tune the model. The maximum accuracy is 74.2% and F1-Score is 63%.

MILESTONE 2

Step 1 & 2: Design, train and test Neural networks, RNN or LSTM classifiers

Using Neural Networks and Language Models

The vectorised form of the text is in 'Description Vector'. This is formed by performing TF-IDF vectorisation. The data is prepared to be placed into the input layer of the neural network.

```
# Setting the Parameters
X = pp_data['Description']
X_V = pp_data['Description_Vector']
Y = pp_data['Accident Level']

max_text_length = X.str.len().max()  # Maximum length of descriptions
vector_dim = len(X_V.iloc[0])  # Length of the first vector

tokenizer = Tokenizer()
tokenizer.fit_on_texts(X)
vocab_size = len(tokenizer.word_index) + 1

embedding_dim = 128
num_classes = 5        # Number of classes
```

SIMPLE NEURAL NETWORK MODEL

```
# Vector branch
vector_input = Input(shape=(vector_dim,), name='vector_input')
vector_dense1 = Dense(64, activation='relu')(vector_input)  # First Dense layer with 64 nodes
vector_dense2 = Dense(16, activation='relu')(vector_dense1)  # Second Dense layer with 16 nodes

# Output layer
output = Dense(num_classes, activation='softmax')(vector_dense2)  # Predict accident level

# Build the model
model_nn = Model(inputs=vector_input, outputs=output)
model_nn.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model_nn.summary()
```

```
Model: "model_4"
__________________________________________________________________
 Layer (type)                Output Shape              Param #
==================================================================
 vector_input (InputLayer)   [(None, 50)]              0
```

```
 dense_12 (Dense)            (None, 64)                3264

 dense_13 (Dense)            (None, 16)                1040

 dense_14 (Dense)            (None, 5)                 85

=================================================================
Total params: 4389 (17.14 KB)
Trainable params: 4389 (17.14 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

RNN MODEL

Both TEXT and VECTORS have been added as Input branches, which are later concatenated as a single input

```
# Text branch
text_input = Input(shape=(max_text_length,), name='text_input')
text_embedding = Embedding(vocab_size, embedding_dim, input_length=max_text_length)(text_input)
text_rnn = SimpleRNN(64)(text_embedding)  # Simple RNN layer

# Vector branch
vector_input = Input(shape=(vector_dim,), name='vector_input')
vector_dense = Dense(64, activation='relu')(vector_input)

# Concatenate branches
concatenated = concatenate([text_rnn, vector_dense])
# Additional Dense layer with 16 nodes
additional_dense = Dense(16, activation='relu')(concatenated)

# Output layer
output = Dense(num_classes, activation='softmax')(additional_dense)

# Build the model
model_rnn = Model(inputs=[text_input, vector_input], outputs=output)
model_rnn.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model_rnn.summary()
```

```
Model: "model_5"
__________________________________________________________________________________________________
 Layer (type)                Output Shape                 Param #   Connected to
==================================================================================================
 text_input (InputLayer)     [(None, 657)]                0         []

 embedding_3 (Embedding)     (None, 657, 128)             376832    ['text_input[0][0]']

 vector_input (InputLayer)   [(None, 50)]                 0         []

 simple_rnn_1 (SimpleRNN)    (None, 64)                   12352     ['embedding_3[0][0]']

 dense_15 (Dense)            (None, 64)                   3264      ['vector_input[0][0]']

 concatenate_3 (Concatenate  (None, 128)                  0         ['simple_rnn_1[0][0]',
 )                                                                     'dense_15[0][0]']

 dense_16 (Dense)            (None, 16)                   2064      ['concatenate_3[0][0]']

 dense_17 (Dense)            (None, 5)                    85        ['dense_16[0][0]']

==================================================================================================
Total params: 394597 (1.51 MB)
Trainable params: 394597 (1.51 MB)
Non-trainable params: 0 (0.00 Byte)
__________________________________________________________________________________________________
```

LSTM MODEL

Both TEXT and VECTORS have been added as Input branches, which are later concatenated as a single input

```
# Text branch
text_input = Input(shape=(max_text_length,), name='text_input')
text_embedding = Embedding(vocab_size, embedding_dim, input_length=max_text_length)(text_input)
text_lstm = LSTM(128, return_sequences=True, dropout=0.2, recurrent_dropout=0.2)(text_embedding)
text_lstm = LSTM(64, dropout=0.3, recurrent_dropout=0.3)(text_lstm)

# Vector branch
```

```
vector_input = Input(shape=(vector_dim,), name='vector_input')
vector_dense = Dense(64, activation='relu')(vector_input)

# Concatenate branches
concatenated = concatenate([text_lstm, vector_dense])
# Additional Dense layer with 16 units
additional_dense = Dense(16, activation='relu')(concatenated)

# Output layer
output = Dense(num_classes, activation='softmax', kernel_regularizer=l2(0.01))(additional_dense)

# Build the model
model_lstm = Model(inputs=[text_input, vector_input], outputs=output)
optimizer = Adam(learning_rate=0.0005)
model_lstm.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model_lstm.summary()
```

```
Model: "model_6"
__________________________________________________________________________________________________
 Layer (type)                Output Shape                 Param #   Connected to
==================================================================================================
 text_input (InputLayer)     [(None, 657)]                0         []

 embedding_4 (Embedding)     (None, 657, 128)             376832    ['text_input[0][0]']

 lstm_4 (LSTM)               (None, 657, 128)             131584    ['embedding_4[0][0]']

 vector_input (InputLayer)   [(None, 50)]                 0         []

 lstm_5 (LSTM)               (None, 64)                   49408     ['lstm_4[0][0]']

 dense_18 (Dense)            (None, 64)                   3264      ['vector_input[0][0]']

 concatenate_4 (Concatenate  (None, 128)                  0         ['lstm_5[0][0]',
 )                                                                   'dense_18[0][0]']

 dense_19 (Dense)            (None, 16)                   2064      ['concatenate_4[0][0]']

 dense_20 (Dense)            (None, 5)                    85        ['dense_19[0][0]']

==================================================================================================
Total params: 563237 (2.15 MB)
Trainable params: 563237 (2.15 MB)
Non-trainable params: 0 (0.00 Byte)
__________________________________________________________________________________________________
```

BI-DIRECTIONAL LSTM MODEL

```
# Text branch
text_input = Input(shape=(max_text_length,), name='text_input')
text_embedding = Embedding(vocab_size, embedding_dim, input_length=max_text_length)(text_input)
text_bilstm = Bidirectional(LSTM(128, return_sequences=True, dropout=0.2, recurrent_dropout=0.2))(text_embedding)
text_bilstm = Bidirectional(LSTM(64, dropout=0.3, recurrent_dropout=0.3))(text_bilstm)

# Vector branch
vector_input = Input(shape=(vector_dim,), name='vector_input')
vector_dense = Dense(64, activation='relu')(vector_input)

# Concatenate branches
concatenated = concatenate([text_bilstm, vector_dense])
# Additional Dense layer with 16 units
additional_dense = Dense(16, activation='relu')(concatenated)

# Output layer
output = Dense(num_classes, activation='softmax', kernel_regularizer=l2(0.01))(additional_dense)

# Build the model
model_bilstm = Model(inputs=[text_input, vector_input], outputs=output)
model_bilstm.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model_bilstm.summary()
```

```
Model: "model_7"
__________________________________________________________________________________________________
 Layer (type)                Output Shape                 Param #   Connected to
==================================================================================================
 text_input (InputLayer)     [(None, 657)]                0         []
```

```
 embedding_5 (Embedding)     (None, 657, 128)             376832    ['text_input[0][0]']

 bidirectional_2 (Bidirecti  (None, 657, 256)             263168    ['embedding_5[0][0]']
 onal)

 vector_input (InputLayer)   [(None, 50)]                 0         []

 bidirectional_3 (Bidirecti  (None, 128)                  164352    ['bidirectional_2[0][0]']
 onal)

 dense_21 (Dense)            (None, 64)                   3264      ['vector_input[0][0]']

 concatenate_5 (Concatenate  (None, 192)                  0         ['bidirectional_3[0][0]',
 )                                                                   'dense_21[0][0]']

 dense_22 (Dense)            (None, 16)                   3088      ['concatenate_5[0][0]']

 dense_23 (Dense)            (None, 5)                    85        ['dense_22[0][0]']

==================================================================================================
Total params: 810789 (3.09 MB)
Trainable params: 810789 (3.09 MB)
Non-trainable params: 0 (0.00 Byte)
__________________________________________________________________________________________________
```

```
# Prepare text data
tokenizer = Tokenizer(num_words=vocab_size)
tokenizer.fit_on_texts(X)
sequences = tokenizer.texts_to_sequences(X)
text_data = pad_sequences(sequences, maxlen=max_text_length)

# Prepare vector data
vector_data = np.array(X_V.tolist())

# Prepare labels
label_encoder = LabelEncoder()
labels = label_encoder.fit_transform(Y)

# Split data into training and test sets
X_train_text, X_test_text, X_train_vector, X_test_vector, y_train, y_test = train_test_split(
    text_data, vector_data, labels, test_size=0.2, random_state=1)
```

Training is done based on the compiled models, with the number of epochs chosen as 20, batch_size as 32 and validation_split as 0.1. The Plain NN model uses only the TF-IDF vectors, while the other models uses both a newly defined vectorisation and the TF-IDF vector as two parallel inputs that are concatenated in the second layer.

FUNCTION TO TRAIN AND EVALUATE THE MODELS

```
def train_and_evaluate(model, X_train_text, X_train_vector, y_train, X_test_text, X_test_vector, y_test, is_nn_model=False)
    if is_nn_model:
        history = model.fit(X_train_vector, y_train, epochs=10, batch_size=32, validation_split=0.1)
        y_pred = model.predict(X_test_vector)
        loss, test_accuracy = model.evaluate(X_test_vector, y_test)
    else:
        history = model.fit([X_train_text, X_train_vector], y_train, epochs=20, batch_size=32, validation_split=0.1)
        y_pred = model.predict([X_test_text, X_test_vector])
        loss, test_accuracy = model.evaluate([X_test_text, X_test_vector], y_test)

    y_pred_labels = np.argmax(y_pred, axis=1)
    avg_training_accuracy = np.mean(history.history['accuracy'])
    precision_recall_report = classification_report(y_test, y_pred_labels, output_dict=True)
    precision = precision_recall_report['weighted avg']['precision']
    recall = precision_recall_report['weighted avg']['recall']

    return avg_training_accuracy, test_accuracy, precision, recall, y_pred_labels

# Training and evaluating each model
print('------------------ STARTING SIMPLE NEURAL NETWORK MODEL ------------------')
avg_train_acc_nn, test_acc_nn, precision_nn, recall_nn, predictions_nn = train_and_evaluate(model_nn, None, X_train_vector,
print('\n------------------ STARTING RNN MODEL ------------------')
avg_train_acc_rnn, test_acc_rnn, precision_rnn, recall_rnn, predictions_rnn = train_and_evaluate(model_rnn, X_train_text, X
print('\n------------------ STARTING LSTM MODEL ------------------')
avg_train_acc_lstm, test_acc_lstm, precision_lstm, recall_lstm, predictions_lstm = train_and_evaluate(model_lstm, X_train_t
print('\n------------------ STARTING BIDIRECTIONAL LSTM MODEL ------------------')
avg_train_acc_bilstm, test_acc_bilstm, precision_bilstm, recall_bilstm, predictions_bilstm = train_and_evaluate(model_bilst
```

```python

# Compute and print confusion matrices
conf_matrix_nn = confusion_matrix(y_test, predictions_nn)
conf_matrix_rnn = confusion_matrix(y_test, predictions_rnn)
conf_matrix_lstm = confusion_matrix(y_test, predictions_lstm)
conf_matrix_bilstm = confusion_matrix(y_test, predictions_bilstm)

print("Confusion Matrix for SIMPLE NEURAL NETWORK Model:\n", conf_matrix_nn)
print("Confusion Matrix for RNN Model:\n", conf_matrix_rnn)
print("Confusion Matrix for LSTM Model:\n", conf_matrix_lstm)
print("Confusion Matrix for BIDIRECTIONAL LSTM Model:\n", conf_matrix_bilstm)
```

```
------------------ STARTING SIMPLE NEURAL NETWORK MODEL ------------------
Epoch 1/10
10/10 [==============================] - 1s 54ms/step - loss: 1.4513 - accuracy: 0.6013 - val_loss: 1.1805 - val_accuracy: 0.8
Epoch 2/10
10/10 [==============================] - 0s 12ms/step - loss: 1.1860 - accuracy: 0.7255 - val_loss: 0.8886 - val_accuracy: 0.8
Epoch 3/10
10/10 [==============================] - 0s 10ms/step - loss: 1.0958 - accuracy: 0.7255 - val_loss: 0.7671 - val_accuracy: 0.8
Epoch 4/10
10/10 [==============================] - 0s 11ms/step - loss: 1.0789 - accuracy: 0.7255 - val_loss: 0.7551 - val_accuracy: 0.8
Epoch 5/10
10/10 [==============================] - 0s 13ms/step - loss: 1.0720 - accuracy: 0.7255 - val_loss: 0.7859 - val_accuracy: 0.8
Epoch 6/10
10/10 [==============================] - 0s 13ms/step - loss: 1.0598 - accuracy: 0.7255 - val_loss: 0.7946 - val_accuracy: 0.8
Epoch 7/10
10/10 [==============================] - 0s 11ms/step - loss: 1.0546 - accuracy: 0.7255 - val_loss: 0.7793 - val_accuracy: 0.8
Epoch 8/10
10/10 [==============================] - 0s 11ms/step - loss: 1.0509 - accuracy: 0.7255 - val_loss: 0.7925 - val_accuracy: 0.8
Epoch 9/10
10/10 [==============================] - 0s 9ms/step - loss: 1.0471 - accuracy: 0.7255 - val_loss: 0.7821 - val_accuracy: 0.882
Epoch 10/10
10/10 [==============================] - 0s 10ms/step - loss: 1.0428 - accuracy: 0.7255 - val_loss: 0.7841 - val_accuracy: 0.8
3/3 [==============================] - 0s 4ms/step
3/3 [==============================] - 0s 5ms/step - loss: 1.0714 - accuracy: 0.7529

------------------ STARTING RNN MODEL ------------------
Epoch 1/20
10/10 [==============================] - 5s 250ms/step - loss: 1.3834 - accuracy: 0.6013 - val_loss: 0.8074 - val_accuracy: 0.8
Epoch 2/20
10/10 [==============================] - 2s 206ms/step - loss: 1.0014 - accuracy: 0.7255 - val_loss: 0.6231 - val_accuracy: 0.8
Epoch 3/20
10/10 [==============================] - 2s 220ms/step - loss: 0.9530 - accuracy: 0.7255 - val_loss: 0.5635 - val_accuracy: 0.8
Epoch 4/20
10/10 [==============================] - 2s 203ms/step - loss: 0.9398 - accuracy: 0.7255 - val_loss: 0.6880 - val_accuracy: 0.8
Epoch 5/20
10/10 [==============================] - 3s 338ms/step - loss: 0.8569 - accuracy: 0.7647 - val_loss: 0.8382 - val_accuracy: 0.8
Epoch 6/20
10/10 [==============================] - 2s 231ms/step - loss: 0.7213 - accuracy: 0.7320 - val_loss: 0.6286 - val_accuracy: 0.8
Epoch 7/20
10/10 [==============================] - 2s 227ms/step - loss: 0.5070 - accuracy: 0.7876 - val_loss: 0.6343 - val_accuracy: 0.8
Epoch 8/20
10/10 [==============================] - 2s 196ms/step - loss: 0.3914 - accuracy: 0.8824 - val_loss: 0.6170 - val_accuracy: 0.8
Epoch 9/20
10/10 [==============================] - 2s 192ms/step - loss: 0.3112 - accuracy: 0.9085 - val_loss: 0.5687 - val_accuracy: 0.8
Epoch 10/20
10/10 [==============================] - 2s 212ms/step - loss: 0.2469 - accuracy: 0.9314 - val_loss: 0.5584 - val_accuracy: 0.8
Epoch 11/20
10/10 [==============================] - 4s 388ms/step - loss: 0.1778 - accuracy: 0.9771 - val_loss: 0.5708 - val_accuracy: 0.8
Epoch 12/20
10/10 [==============================] - 2s 211ms/step - loss: 0.1275 - accuracy: 0.9869 - val_loss: 0.5841 - val_accuracy: 0.8
Epoch 13/20
10/10 [==============================] - 2s 195ms/step - loss: 0.0958 - accuracy: 0.9869 - val_loss: 0.6109 - val_accuracy: 0.8
Epoch 14/20
10/10 [==============================] - 2s 212ms/step - loss: 0.0746 - accuracy: 0.9804 - val_loss: 0.6256 - val_accuracy: 0.8
Epoch 15/20
10/10 [==============================] - 2s 216ms/step - loss: 0.0621 - accuracy: 0.9804 - val_loss: 0.6320 - val_accuracy: 0.8
Epoch 16/20
10/10 [==============================] - 2s 224ms/step - loss: 0.0590 - accuracy: 0.9804 - val_loss: 0.6391 - val_accuracy: 0.8
```

```python
metrics_nn = [avg_train_acc_nn, test_acc_nn, precision_nn, recall_nn]
metrics_rnn = [avg_train_acc_rnn, test_acc_rnn, precision_rnn, recall_rnn]
metrics_lstm = [avg_train_acc_lstm, test_acc_lstm, precision_lstm, recall_lstm]
metrics_bilstm = [avg_train_acc_bilstm, test_acc_bilstm, precision_bilstm, recall_bilstm]

# Create a DataFrame
metrics_df = pd.DataFrame({
    'Model': ['NN', 'RNN', 'LSTM', 'BiLSTM'],
    'Avg Training Accuracy': [metrics_nn[0], metrics_rnn[0], metrics_lstm[0], metrics_bilstm[0]],
    'Test Accuracy': [metrics_nn[1], metrics_rnn[1], metrics_lstm[1], metrics_bilstm[1]],
    'Precision': [metrics_nn[2], metrics_rnn[2], metrics_lstm[2], metrics_bilstm[2]],
```

```
    'Recall': [metrics_nn[3], metrics_rnn[3], metrics_lstm[3], metrics_bilstm[3]]
})

metrics_df
```

	Model	Avg Training Accuracy	Test Accuracy	Precision	Recall
0	NN	0.713072	0.752941	0.566920	0.752941
1	RNN	0.881046	0.611765	0.543791	0.611765
2	LSTM	0.580882	0.752941	0.566920	0.752941
3	BiLSTM	0.857680	0.658824	0.633619	0.658824

The performance of the various models are presented in a dataframe.

Obervations:

- The Training accuracy is highest but testing accuracy has dropped with RNN model.
- The Simple Neural Network model is performing the best for testing data but has lowest accuracy on Training data.
- As the trainig accuracy is dropping for models, the testing accuracy is increasing respectively.
- Precision is almost similar with all the models, with highest being in LSTM model.
- Compared to basic ML classifiers, Neural Network models are performing better on minority classes but it's significantly low, hence **we need to perform Data Augmentation to balance the dataset**.

Balancing the class distribution in the dataset using NLP Augmentation techniques

```
X = pp_data['Description']
Y = pp_data['Accident Level']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3, random_state=42)
```

```
Y_train.value_counts()
```

```
I      216
II      28
IV      24
III     24
V        5
Name: Accident Level, dtype: int64
```

```
minority_classes = ['II', 'III', 'IV', 'V']
```

SYNONYM AUGMENTATION

```
aug_syn = naw.SynonymAug(aug_src='wordnet', aug_p=0.1)
```

```
aug_syn_desc = []
aug_syn_lab = []

for i, label in enumerate(Y_train):
    if label in minority_classes:
        aug_syn_text = aug_syn.augment(X_train.iloc[i])
        aug_syn_desc.append(aug_syn_text)
        aug_syn_lab.append(label)

aug_syn_data = pd.DataFrame({'Description': aug_syn_desc, 'Accident_Level': aug_syn_lab})

X_train_aug_syn = pd.concat([X_train, aug_syn_data['Description']])
Y_train_aug_syn = pd.concat([Y_train, aug_syn_data['Accident_Level']])
```

```
Y_train_aug_syn.value_counts()
```

```
I      216
II      56
IV      48
III     48
V       10
dtype: int64
```

```
print('Shape of Dataset BEFORE Augmentation: (', X_train.shape[0], ',', Y_train.shape[0], ')')
print('Shape of Dataset AFTER Augmentation: (', X_train_aug_syn.shape[0], ',', Y_train_aug_syn.shape[0], ')')
```

```
Shape of Dataset BEFORE Augmentation: ( 297 , 297 )
Shape of Dataset AFTER Augmentation: ( 378 , 378 )
```

CONTEXTUAL WORD EMBEDDINGS AUGMENTATION USING BERT GENERATIVE MODEL

```
aug_contextual = naw.ContextualWordEmbsAug(model_path='bert-base-uncased', action="substitute")

aug_contextual_desc = []
aug_contextual_lab = []

for i, label in enumerate(Y_train):
    if label in minority_classes:
        aug_contextual_text = aug_contextual.augment(X_train.iloc[i])
        aug_contextual_desc.append(aug_contextual_text)
        aug_contextual_lab.append(label)

aug_contextual_data = pd.DataFrame({'Description': aug_contextual_desc, 'Accident_Level': aug_contextual_lab})

# Combine the original training data with the augmented data
X_train_aug = pd.concat([X_train, aug_syn_data['Description'], aug_contextual_data['Description']])
Y_train_aug = pd.concat([Y_train, aug_syn_data['Accident_Level'], aug_contextual_data['Accident_Level']])
```

```
tokenizer_config.json: 100%                28.0/28.0 [00:00<00:00, 1.17kB/s]
config.json: 100%                          570/570 [00:00<00:00, 22.3kB/s]
vocab.txt: 100%                            232k/232k [00:00<00:00, 4.02MB/s]
tokenizer.json: 100%                       466k/466k [00:00<00:00, 14.6MB/s]
model.safetensors: 100%                    440M/440M [00:05<00:00, 110MB/s]
```

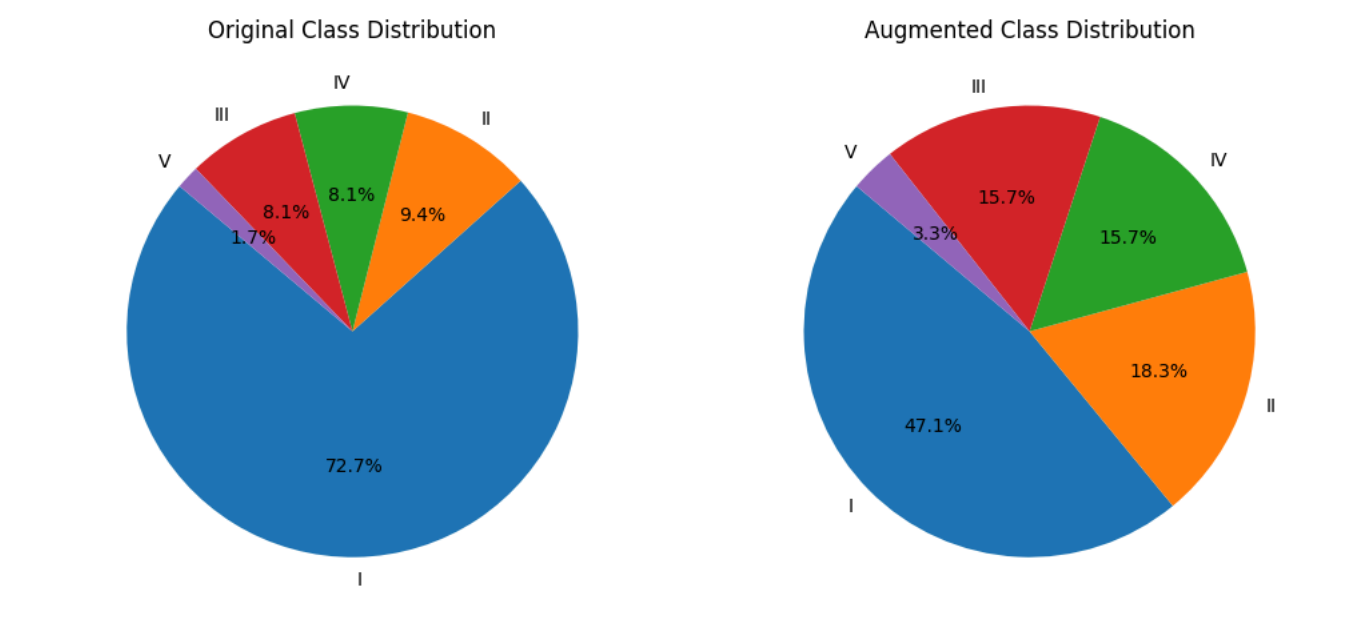
```
Y_train_aug.value_counts()
```

```
I      216
II      84
IV      72
III     72
V       15
dtype: int64
```

```
print('Shape of Training Dataset BEFORE Augmentation: ', X_train.shape)
print('Shape of Training Dataset AFTER Augmentation: ', X_train_aug.shape)
print('Shape of Testing Dataset BEFORE Augmentation: ', X_test.shape)
```

```
Shape of Training Dataset BEFORE Augmentation:  (297,)
Shape of Training Dataset AFTER Augmentation:  (459,)
Shape of Testing Dataset BEFORE Augmentation:  (128,)
```

```
## Pie Chart for Class Distribution Before and After Augumentation

fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].pie(Y_train.value_counts(), labels=Y_train.value_counts().keys(), autopct='%1.1f%%', startangle=140)
axes[0].set_title('Original Class Distribution')
axes[1].pie(Y_train_aug.value_counts(), labels=Y_train_aug.value_counts().keys(), autopct='%1.1f%%', startangle=140)
axes[1].set_title('Augmented Class Distribution')
plt.show()
```



Performing Vectorization over the augmented training data and testing data

The Data Augmentation techniques used above are based on similar words and context of the accidents that have occurred in minority classes. TF-IDF Vectorization technique focus on the importance of terms in documents, which will not work with these Data Augmentation techniques. Hence, we have chosen **GLOVE EMBEDDINGS** as the Vecotrization technique, which is more effective for use cases involving semantic understanding, word similarity, and capturing relationships between words.

```
# Load GloVe embeddings
glove = vocab.GloVe(name='6B', dim=50)

# Function to get word vector, handling out-of-vocabulary words
def get_word_vector(word):
    try:
        return glove[word]
    except KeyError:
        return torch.zeros_like(glove['the'])

# Create GloVe embeddings for the training data
X_train_aug_glove = []
for description in X_train_aug.index.astype(str):
    description_vector = [get_word_vector(word) for word in description.split()]
    mean_vector = torch.stack(description_vector).mean(dim=0).tolist()
    X_train_aug_glove.append(mean_vector)

# Create GloVe embeddings for the testing data
X_test_glove = []
for description in X_test:
    description_vector = [get_word_vector(word) for word in description.split()]
    mean_vector = torch.stack(description_vector).mean(dim=0).tolist()
    X_test_glove.append(mean_vector)
```

```
.vector_cache/glove.6B.zip: 862MB [02:39, 5.39MB/s]
100%|██████████| 399999/400000 [00:15<00:00, 25114.84it/s]
```

```
X_train_aug_glove_df = pd.DataFrame({'Description': X_train_aug, 'Description_Glove': X_train_aug_glove})
X_test_glove_df = pd.DataFrame({'Description': X_test, 'Description_Glove': X_test_glove})
```

```
X_train_aug_glove_df.head()
```

	Description	Description_Glove
250	igniting furnace battery reflux hot gas reaching face employee	[0.7232499718666077, 0.23351000249385834, 1.3867000341415405, -0.41631001234054565, 0.1398800015449524, -0.5298299789428711, 0.15956999361515045, -0.6353999972343445, 0.12193000316619873, -0.022370999678969383, -0.696150004863739, -1.2218999862670898, 0.6085600256919861, -0.8629500269889832, 0.670520007610321, -0.6866599917411804, 0.08904799818992615, 1.12909996509552, -0.9315099716186523, -0.5867699980735779, 0.44631001353263855, -0.014991000294685364, 0.27046999335289, -0.009825799614191055, -0.4528200030326843, -0.35826998949050903, 0.1788100004196167, -0.5435299873352051, -0.7968000173568726, -0.17353999614715576, 2.733099937438965, 0.09774400293827057, 0.14858999848365784, 1.0835000276565552, 0.597029983997345, -0.1004600003361702, 0.3175100088119507, -0.3446899950504303, 0.20927999913692474, 0.2311200052499771, 0.0736280009150505, 0.10831999778747559, 1.2170000076293945, 0.2884199917316437, -0.5393700003623962, -0.004199199844151735, -0.34918999671936035, 0.2098499983549118, -0.602590024471283, -0.3149400055408478]
397	moment collaborator carried inspection conveyor belt tail pulley height load polymer maslucan collaborator heard noise note belt moving towards tail pulley fragmentos mineral fragment projected towards access ramp impacting collaborator evacuated medical post	[0.37419000267982483, -0.5891500115394592, 0.9552099704742432, -0.3074600100517273, 0.3250499963760376, 0.12139999866485596, 0.5072900056838989, -0.5068600177764893, -0.1680999994277954, -0.14016999304294586, -0.9163100123405457, -0.6151000261306763, 0.18024000525474548, -1.0396000146865845, 0.23739999532699585, -0.9955400228500366, -0.8052099943161011, 0.18095999956130981, -0.44176000356674194, 0.7800300121307373, -0.08721599727869034, -0.4574599862098694, 0.7172499895095825, -0.36983999609947205, 0.08961600065231323, 0.58992999792099, 0.3482399880886078, -0.4031200110912323, -0.26280999183654785, -0.24598999321460724, -0.0995749980211258, 0.2586899995803833, 0.8070399761199951, 1.0032000541687012, 0.7045000195503235, -0.41286998987197876, 0.6037999987602234, 0.28916001319885254, -0.045875001698732376, 0.4345700144767761, -0.43580999970436096, -0.64860999584198, 0.9506999850273132, 0.29124000668525696, -1.0049999952316284, 0.6634399890899658, -0.26941001415252686, 0.2857699990272522, 0.2578200101852417, -0.15658000111579895]
		[0.0633779987692833, 0.10044000297784805, 1.0613000392913818, -0.4291200041770935, 0.3640899956226349, -0.11660999804735184, 0.3280600011348724 4, -0.2161899954080581 7

Training above Neural Network models on augmented training data, and Evaluating them

```
# Concatenate training and testing datasets
X_glove_df = pd.concat([X_train_aug_glove_df, X_test_glove_df], ignore_index=True)

# Setting the Parameters
X = X_glove_df['Description']
X_V = X_glove_df['Description_Glove']
Y = pp_data['Accident Level']

max_text_length = X.str.len().max()  # Maximum length of descriptions
vector_dim = len(X_V.iloc[0])  # Length of the first vector

tokenizer = Tokenizer()
tokenizer.fit_on_texts(X)
vocab_size = len(tokenizer.word_index) + 1

embedding_dim = 128
num_classes = 5        # Number of classes
```

```
vector_dim
```

```
50
```

```
max_text_length
```

```
657
```

```
# Prepare training text data
tokenizer = Tokenizer(num_words=vocab_size)
tokenizer.fit_on_texts(X_train_aug_glove_df['Description'])
sequences = tokenizer.texts_to_sequences(X_train_aug_glove_df['Description'])
X_train_text = pad_sequences(sequences, maxlen=max_text_length)

# Prepare testing text data
tokenizer = Tokenizer(num_words=vocab_size)
tokenizer.fit_on_texts(X_test_glove_df['Description'])
sequences = tokenizer.texts_to_sequences(X_test_glove_df['Description'])
X_test_text = pad_sequences(sequences, maxlen=max_text_length)

# Prepare vector data
X_train_vector = np.array(X_train_aug_glove_df['Description_Glove'].tolist())
X_test_vector = np.array(X_test_glove_df['Description_Glove'].tolist())

# Prepare labels
label_encoder = LabelEncoder()
y_train = label_encoder.fit_transform(Y_train_aug)
y_test = label_encoder.fit_transform(Y_test)
```

```
len(X_train_text)
```

```
459
```

```
len(X_train_vector)
```

```
459
```

```
len(y_train)
```

```
459
```

TRAINING AND EVALUATING THE NEURAL NETWORKS

```
### The Simple NN model and RNN model are run through a simple grid search varying the learning rates.

from tensorflow.keras.models import clone_model
from tensorflow.keras.optimizers import Adam

## Gridsearch and Report for Simple NN
def perform_grid_search_nn(base_model, param_grid, X_train, y_train, X_test, y_test):
    results = []

    for params in param_grid:
        model = clone_model(base_model)
        optimizer = Adam(learning_rate=params['learning_rate'])
        model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

        avg_train_acc_nn, test_acc_nn, precision_nn, recall_nn, predictions_nn = train_and_evaluate(model_nn, None, X_trair

        results.append({
            'params': params,
            'avg_train_accuracy': avg_train_acc_nn,
            'test_accuracy': test_acc_nn,
            'precision': precision_nn,
            'recall': recall_nn
        })

    return results

# Parameters
param_grid = [
    # Adam optimizer with different learning rates
    {'optimizer': 'adam', 'learning_rate': 0.001 },
    {'optimizer': 'adam', 'learning_rate': 0.0001 }
]

grid_search_results = perform_grid_search_nn(model_nn, param_grid, X_train_vector, y_train, X_test_vector, y_test)

# Dataframe the results
```

```
grid_search_results_df = pd.DataFrame(grid_search_results)
df_params = grid_search_results_df['params'].apply(pd.Series)
grid_search_results_df = pd.concat([grid_search_results_df.drop(['params'], axis=1), df_params], axis=1)
grid_search_results_df

## Gridsearch and Report for RNN
def perform_grid_search_rnn(base_model, param_grid, X_train_text, X_train_vector, y_train, X_test_text, X_test_vector, y_te
    results = []

    for params in param_grid:
        model = clone_model(base_model)
        optimizer = Adam(learning_rate=params['learning_rate'])
        model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

        avg_train_acc_rnn, test_acc_rnn, precision_rnn, recall_rnn, predictions_rnn = train_and_evaluate(model_rnn, X_trair

        results.append({
            'params': params,
            'avg_train_accuracy': avg_train_acc_rnn,
            'test_accuracy': test_acc_rnn,
            'precision': precision_rnn,
            'recall': recall_rnn
        })

    return results

# Parameters
param_grid = [
    # Adam optimizer with different learning rates
    {'optimizer': 'adam', 'learning_rate': 0.001 },
    {'optimizer': 'adam', 'learning_rate': 0.0001 }
]

grid_search_results = perform_grid_search_rnn(model_rnn, param_grid, X_train_text, X_train_vector, y_train, X_test_text, X_

# Dataframe the results
grid_search_results_df = pd.DataFrame(grid_search_results)
df_params = grid_search_results_df['params'].apply(pd.Series)
grid_search_results_df = pd.concat([grid_search_results_df.drop(['params'], axis=1), df_params], axis=1)
grid_search_results_df
```

```
Epoch 1/10
13/13 [==============================] - 1s 21ms/step - loss: 1.4114 - accuracy: 0.4286 - val_loss: 1.9009 - val_accuracy: 0.0
Epoch 2/10
13/13 [==============================] - 0s 6ms/step - loss: 1.2532 - accuracy: 0.5254 - val_loss: 1.9654 - val_accuracy: 0.00
Epoch 3/10
13/13 [==============================] - 0s 5ms/step - loss: 1.2242 - accuracy: 0.5617 - val_loss: 1.8427 - val_accuracy: 0.00
Epoch 4/10
13/13 [==============================] - 0s 6ms/step - loss: 1.2049 - accuracy: 0.5642 - val_loss: 1.7527 - val_accuracy: 0.00
Epoch 5/10
13/13 [==============================] - 0s 8ms/step - loss: 1.1925 - accuracy: 0.5617 - val_loss: 1.7526 - val_accuracy: 0.00
Epoch 6/10
13/13 [==============================] - 0s 8ms/step - loss: 1.1784 - accuracy: 0.5642 - val_loss: 1.7476 - val_accuracy: 0.00
Epoch 7/10
13/13 [==============================] - 0s 6ms/step - loss: 1.1693 - accuracy: 0.5642 - val_loss: 1.7414 - val_accuracy: 0.00
Epoch 8/10
13/13 [==============================] - 0s 8ms/step - loss: 1.1639 - accuracy: 0.5642 - val_loss: 1.7112 - val_accuracy: 0.00
Epoch 9/10
```

```
## Gridsearch and Report for RNN
def perform_grid_search_rnn(base_model, param_grid, X_train_text, X_train_vector, y_train, X_test_text, X_test_vector, y_te
    results = []

    for params in param_grid:
        model = clone_model(base_model)
        optimizer = Adam(learning_rate=params['learning_rate'])
        model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

        avg_train_acc_rnn, test_acc_rnn, precision_rnn, recall_rnn, predictions_rnn = train_and_evaluate(model_rnn, X_trair

        results.append({
            'params': params,
            'avg_train_accuracy': avg_train_acc_rnn,
            'test_accuracy': test_acc_rnn,
            'precision': precision_rnn,
            'recall': recall_rnn
        })

    return results

# Parameters
param_grid = [
    # Adam optimizer with different learning rates
    {'optimizer': 'adam', 'learning_rate': 0.001 },
    {'optimizer': 'adam', 'learning_rate': 0.0001 }
]

grid_search_results = perform_grid_search_rnn(model_rnn, param_grid, X_train_text, X_train_vector, y_train, X_test_text, X_

# Dataframe the results
grid_search_results_df = pd.DataFrame(grid_search_results)
df_params = grid_search_results_df['params'].apply(pd.Series)
grid_search_results_df = pd.concat([grid_search_results_df.drop(['params'], axis=1), df_params], axis=1)
grid_search_results_df
```

```
Epoch 7/20
```

```
print('------------------ STARTING SIMPLE NEURAL NETWORK MODEL ------------------')
avg_train_acc_nn, test_acc_nn, precision_nn, recall_nn, predictions_nn = train_and_evaluate(model_nn, None, X_train_vector,
print('\n------------------ STARTING RNN MODEL ------------------')
```